

NAS FOR OPTIMIZED EDGE MODELS IN WEARABLE HEALTH MONITORING

A PROJECT REPORT

Submitted by

BALA VIGNESH N (727823TUAM006)

SRADHAA O (727823TUAM048)

TAMILARASU R (727823TUAM056)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

SRI KRISHNA COLLEGE OF TECHNOLOGY

(An Autonomous Institution)



(Approved by AICTE and Affiliated to Anna University, Chennai)

Accredited by NAAC with 'A' Grade

Coimbatore – 641 042

ANNA UNIVERSITY : CHENNAI 600 025

SEPTEMBER 2026



SRI KRISHNA COLLEGE OF TECHNOLOGY

(An Autonomous Institution)

Affiliated to Anna University and Approved by AICTE

KOVAIPUDUR, COIMBATORE – 641 042

BONAFIDE CERTIFICATE

Certified that this project report “**NAS FOR OPTIMIZED EDGE MODELS IN WEARABLE HEALTH MONITORING**” is the bonafide work of “**BALA VIGNESH N, SRADHAA O and TAMILARASU R**” who carried out the project work under my supervision.

SIGNATURE

Dr. SUMA SIRA JACOB,
HEAD OF THE DEPARTMENT
Department of CSE (Artificial
Intelligence and Machine Learning)
Sri Krishna College of Technology
Coimbatore – 641 042.

SIGNATURE

Ms. S. SATHYA BAMA,
SUPERVISOR
Assistant Professor
Department of CSE (Artificial
Intelligence and Machine Learning)
Sri Krishna College of Technology
Coimbatore – 641 042.

Submitted for the Project Viva-Voce Examination held on

_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER



SRI KRISHNA COLLEGE OF TECHNOLOGY

(An Autonomous Institution)

Affiliated to Anna University and Approved by AICTE

KOVAIPUDUR, COIMBATORE – 641 042

DECLARATION

We affirm that the project report titled “**NAS FOR OPTIMIZED EDGE MODELS IN WEARABLE HEALTH MONITORING**” being submitted in partial fulfilment for the award of **BACHELOR OF ENGINEERING in COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)** is the original work carried out by us. It has not formed part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Name	Reg. No.	Signature
BALA VIGNESH N	727823TUAM006	
SRADHAA O	727823TUAM048	
TAMILARASU R	727823TUAM056	

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Name and Signature of the Supervisor with Date

ACKNOWLEDGEMENT

First and foremost, we thank the Almighty for being our guiding light and for showering His gracious blessings throughout the course of this project.

We express our heartfelt gratitude to our beloved Principal, **Dr. M. G. Sumithra**, for her kindness, encouragement, and unwavering support throughout the project work.

With a grateful heart, we extend our sincere thanks to our Program Coordinator, **Dr. Suma Sira Jacob**, Department of Computer Science and Engineering (AI& ML), for her continuous motivation, guidance, and valuable support in the successful completion of our project.

We express our sincere gratitude to our Project Coordinator and Project Supervisor, **Ms. S. Sathya Bama**, Assistant Professor, Department of Computer Science and Engineering (AI & ML), for her valuable encouragement, constant guidance, constructive suggestions, and continuous assistance throughout the course of the project. Her support and motivation greatly contributed to enhancing our knowledge and successfully completing this project.

We express our sincere thanks to all the **Teaching and Non-Teaching Staff** of the Department of CSE (AI & ML) for their direct and indirect contributions towards the completion of this project.

Finally, we extend our heartfelt thanks to our **Family members and Beloved friends** for their continuous support, patience, and encouragement in all our endeavour.

ABSTRACT

Continuous wearable electrocardiogram (ECG) monitoring is difficult to achieve in practice, since arrhythmia classifiers that are accurate enough to be clinically useful typically demand more memory and computation than an edge device can spare. This project addresses that gap through a resource-aware Neural Architecture Search (NAS) framework built for real-time wearable ECG arrhythmia monitoring. Single-lead ECG signals are captured by an AD8232 analog front end wired to an ESP32, which streams the sampled data over Wi-Fi to an edge-based processing pipeline. There, the signal passes through high-pass, notch and band-pass filtering before being segmented into 150-sample beat windows centred on each detected R-peak. Every input pairs the normalized ECG waveform with two causal RR-context features, yielding a 150×3 tensor. Candidate architectures are then searched evolutionarily and scored on macro-F1, parameter count and inference latency, with pre-pruning enforcing a hard 120,000-parameter ceiling. The architecture that emerges from this search uses just 1,269 trainable parameters and is compressed further through full-integer INT8 quantization. On 70,396 held-out beats, the quantised model reaches 0.8899 accuracy and 0.6188 macro-F1, with an estimated runtime memory footprint of only 10,869 bytes. Together, these results point toward compact, resource-efficient ECG intelligence for wearable edge healthcare. By combining architecture-level optimization with quantization, the approach lowers deployment cost without sacrificing clinically relevant classification quality, and its resource-aware design leaves room for future work in which measured hardware latency and power consumption feed directly back into the NAS objective.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	iv
	ABSTRACT	v
	LIST OF TABLES	ix
	LIST OF FIGURES	x
	LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	xi
1	INTRODUCTION	1
	1.1 Background	2
	1.2 Neural Architecture Search for Edge ECG Classification	3
	1.3 Motivation	4
	1.4 Organization Of The Report	5
2	LITERATURE REVIEW	7
	2.1 Neural Architecture Search	7
	2.2 Lightweight Deep Learning For ECG Classification	8
	2.3 Multi-Objective Neural Architecture Search	8
	2.4 Latency-Aware And Hardware-Aware Nas	9
	2.5 Power And Energy-Aware Edge Computing	10
	2.6 Quantization And Resource-Efficient Deployment	11
	2.7 Summary Of Literature Review	11
3	PROBLEM IDENTIFICATION	13

	3.1 Problem Statement	13
	3.2 Limitations Of Existing Systems	14
	3.3 Research Gap	15
4	OBJECTIVE	17
	4.1 Objectives Of The Project	17
	4.2 Scope Of The Project	18
5	PROBLEM SOLUTION	21
	5.1 Proposed System Overview	21
	5.2 Working Principle	22
	5.3 Multi-Objective Optimization Model	23
	5.4 Pareto-Based Architecture Selection	25
	5.5 Latency, Power And Resource-Efficiency Model	26
	5.6 Selected Architecture And Quantization	27
	5.7 System Flow And Solution Summary	28
6	BLOCK DIAGRAM	31
	6.1 Overall System Architecture	31
	6.2 Description Of Functional Units	34
	6.3 System Flow	35
	6.4 Nas Architecture	38
7	METHODOLOGY	41
	7.1 ECG Data Preparation And Input Representation	41
	7.2 Nas Search Space	41
	7.3 Evolutionary Nas Search	43
	7.4 Multi-Objective Evaluation	44
	7.5 Pareto Analysis And Weighted Fitness	45

	7.6 Selected Architecture And Full Training	46
	7.7 Int8 Quantization	47
	7.8 Performance And Resource Evaluation	47
	7.9 Deployment Workflow	48
8	RESULTS AND DISCUSSION	51
	8.1 Experimental Setup	51
	8.2 Nas Search And Architecture Selection	52
	8.3 Classification Performance	52
	8.4 Class-Wise Performance Analysis	53
	8.5 Memory And Model Size Analysis	54
	8.6 Inference Latency Analysis	55
	8.7 Effect Of Int8 Quantization	56
	8.8 Accuracy, Efficiency And Deployment Tradeoff	57
	8.9 Discussion	58
9	CONCLUSION	60
	9.1 Conclusion	60
	9.2 Future Scope	62
	APPENDICES	64
	SDG MAPPING	68
	REFERENCES	70

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
7.1	NAS Search-Space Parameters	42
7.2	Evolutionary NAS Configuration	43
7.3	Selected NAS Architecture	46
7.4	Classification Performance Comparison	47
7.5	Memory and Latency Comparison of NAS Model Variants	48
8.1	Class-wise Performance of NAS INT8 Model	53
8.2	Memory and Model Size Comparison	54
8.3	Inference Latency Comparison	55
8.4	Overall NAS Performance and Deployment Characteristics	57

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
6.1	Overall system architecture of the proposed multi-objective NAS-based wearable ECG arrhythmia classification system	32
6.2	System flow algorithm proposed for the multi-objective NAS based wearable ECG classification system	36
6.3	Selected NAS architecture for wearable ECG arrhythmia classification	39
8.1	Class-wise Performance Analysis of NAS	53
8.2	Model Size with Memory Comparison	54
8.3	Inference Latency Comparison	55

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
CNN	Convolutional Neural Network
ECG	Electrocardiogram
ESP32	Espressif Systems 32-bit Microcontroller
F1	F1-Score
FP32	32-bit Floating Point
GAP	Global Average Pooling
GPIO	General Purpose Input/Output
IoT	Internet of Things
INT8	8-bit Integer
LSTM	Long Short-Term Memory
MCU	Microcontroller Unit
NAS	Neural Architecture Search
NN	Neural Network
RAM	Random Access Memory
ReLU	Rectified Linear Unit
ReLU6	Rectified Linear Unit capped at 6
RNN	Recurrent Neural Network
RR	R-R Interval
TFLM	TensorFlow Lite for Microcontrollers
MIT-BIH	Massachusetts Institute of Technology – Beth Israel Hospital
SDG	Sustainable Development Goal

SYMBOLS

SYMBOL	DESCRIPTION
A	Candidate neural-network architecture
K	Number of ECG classes
$F1_k$	F1-score of class k
P_k	Precision of class k
R_k	Recall of class k
$F1_macro$	Macro-F1 score
N_{params}	Number of trainable parameters
L	Inference latency
w_1	Weight assigned to classification performance
w_2	Weight assigned to parameter cost
w_3	Weight assigned to inference latency
T	Length of the temporal/context window
X_t	Input sequence at time t
x_t	Feature vector at time t
I	ECG input tensor
C	Number of convolutional stages
F	Number of convolutional filters
K_e	Convolution kernel length
S	Convolution stride
D	Dropout rate
H	Dense-layer width
M	Model memory requirement
E_{inf}	Energy consumed per inference
P_{avg}	Average device power

CHAPTER 1

INTRODUCTION

This chapter sets the stage for the rest of the report. It covers the background of wearable ECG monitoring, why real-time arrhythmia classification is difficult on resource-constrained edge devices, how Neural Architecture Search (NAS) helps design compact networks, the motivation behind a resource-aware approach, and how the remaining chapters are organized.

Continuous ECG monitoring matters for intelligent healthcare precisely because cardiac rhythm abnormalities are often intermittent and can easily be missed during a brief clinical visit. Wearable sensing closes this gap by letting ECG signals be captured continuously, outside the clinic. A compact platform built from a single-lead ECG front end, a wireless-enabled microcontroller and a small battery can gather this physiological data over long stretches of time [1]. Capturing the signal, however, is only half the problem: interpreting the waveform in real time calls for computational models that are often too heavy to run on such resource-constrained hardware [2].

Conventional ECG classifiers tend to be demanding enough to need server-class hardware, which pushes wearable systems toward one of two compromises: send raw physiological data to a remote platform, or fall back to simpler on-device measurements such as heart-rate estimation. The first option adds latency, communication overhead and privacy risk; the second sacrifices diagnostic depth, since limited local processing caps how much information can be extracted directly on the wearable [3]. Both point

to the same need - compact, efficient models capable of ECG beat classification under realistic edge-computing constraints.

1.1 BACKGROUND

Automated ECG analysis is not a new problem; it has been studied for decades. Earlier work generally combined signal-processing techniques for QRS detection with classifiers built on morphological and rhythm-related features. Deep learning changed this by letting one-dimensional convolutional networks learn ECG morphology directly from segmented signals, cutting down the need for hand-engineered features [4]. Even so, many of these high-performing models were not designed with embedded deployment constraints in mind [5].

Model size and memory usage are tightly constrained on wearable and edge platforms [6]. Once application software and communication routines have claimed their share of memory, what remains for neural-network weights can be quite small. Activation memory adds a further constraint, since intermediate input and output tensors often need to coexist during inference. Embedded microcontrollers can also have limited floating-point support, which makes integer-based computation an attractive option for resource-constrained deployment [7]. For all these reasons, the network architecture cannot be designed in isolation from the computational and memory budget available to it.

The monitoring system built for this project pairs an AD8232-based single-lead ECG acquisition stage with an ESP32 microcontroller. The ESP32 handles signal acquisition, packetisation, status display and wireless transmission, while an edge-side pipeline takes care of signal conditioning, beat segmentation and classification. Before beat-level analysis begins, the acquired signals are resampled to a common 250 Hz rate and passed through high-pass, notch and band-pass filtering stages.

Each detected heartbeat is captured as a fixed 150-sample window centred on its R-peak, and the classifier input is built as a 150×3 tensor combining the normalized ECG waveform with two causal RR-context features - a prematurity ratio and an absolute heart-rate-related term. These rhythm features are not optional extras: certain arrhythmia types, supraventricular ectopic beats in particular, are hard to separate from normal beats on morphology alone.

1.2 NEURAL ARCHITECTURE SEARCH FOR EDGE ECG CLASSIFICATION

Neural Architecture Search automates what used to be a manual exercise - choosing the number of convolutional stages, filter counts, kernel sizes, strides, pooling operations and dense-layer dimensions - and instead treats architecture selection as an optimization problem [8]. Early NAS work leaned on reinforcement learning; later methods turned to evolutionary and differentiable techniques to bring the search cost down [9]. Resource-aware variants go a step further, folding hardware or model-resource considerations directly into the selection process [10].

For this project, the search space is a family of one-dimensional convolutional networks, with candidates varying in the number of stages, filter count, kernel length, stride, pooling, dropout, activation type and dense-head width. An evolutionary strategy - tournament selection, mutation and single-point crossover - drives the search, starting from an initial population of ten candidates. Before any candidate reaches the expensive proxy-training stage, architecture-space pre-pruning discards models that exceed the 120,000-parameter constraint.

The search follows a multi-objective formulation: macro-F1 is maximized while parameter count and inference latency are pushed down. Macro-F1 is chosen over plain accuracy because the ECG classes are

imbalanced, and macro averaging gives minority classes their fair share of weight during selection. Candidates are compared on the Pareto front, with weighted objective terms used to break ties among competing architectures.

What the search settled on is a compact design: two convolutional stages, global average pooling, a 32-unit dense layer and a five-class softmax output. The first convolution uses 16 filters with a kernel length of 7 and stride 2; the second uses 8 filters with a kernel length of 3 and stride 2. Both convolutional stages use ReLU6 activation with dropout, and the resulting architecture holds only 1,269 trainable parameters.

1.3 MOTIVATION

These observations set the direction for the project: closing the gap between accurate ECG arrhythmia classification and the practical limits of wearable edge hardware. The core idea is to let NAS discover, on its own, an architecture that fits within a predefined resource budget while still classifying usefully. Wearable ECG acquisition, numerical signal conditioning, causal rhythm-context representation, evolutionary architecture search and integer quantization are brought together into one end-to-end framework to achieve this.

There is also a more general motivation behind this: manually designed neural networks have their limits [11]. In the experiments reported here, NAS converged on a model roughly eleven times smaller than the conventional baseline while matching its balanced classification performance. That the search chose global average pooling and progressively fewer filters across convolutional stages suggests that automated search can land on compact configurations a manual design process might easily overlook.

A further motivation is the need to evaluate minority arrhythmia classes rather than leaning on overall accuracy alone. The final quantised model performs strongly on the Normal and Ventricular classes, while Fusion remains a harder case. The study also shows that preserving minority-class representation during NAS proxy validation matters, since an unrepresentative proxy can steer the search toward a weaker architecture.

The project therefore aims to build a foundation for efficient intelligent ECG monitoring in which architecture, memory, latency and deployment constraints are weighed together rather than one at a time.

1.4 ORGANIZATION OF THE REPORT

The rest of the report is organized as follows. Chapter 2 reviews literature on wearable ECG monitoring, automated arrhythmia classification, efficient neural networks, Neural Architecture Search, model compression and quantization. Chapter 3 identifies the problem this work addresses and outlines the limitations of conventional ECG classification approaches on resource-constrained edge systems. Chapter 4 lays out the objectives of the proposed system, covering compact architecture discovery, real-time classification and resource-aware deployment. Chapter 5 describes the proposed solution, including ECG acquisition, signal conditioning, beat segmentation, input representation and the NAS framework. Chapter 6 presents the block diagram and overall system flow. Chapter 7 walks through the methodology - dataset preparation, evolutionary search, proxy validation, model training, quantization and deployment. Chapter 8 presents the experimental results and discussion, covering classification performance, model size, memory footprint and latency. Chapter 9 concludes the report and discusses future

directions such as direct ESP32 deployment, target-hardware benchmarking and broader deployment objectives.

Design decisions throughout the report are tied back to measurable system requirements - classification performance, parameter count, memory footprint and inference latency. The experimental results are laid out so the connection between the original deployment constraints, the architecture-search process and the final model characteristics stays traceable. In short, the report moves from the healthcare and edge-computing problem, to the proposed NAS-based solution, and finally to its experimental verification.

Chapter 1 lays out the case for resource-aware intelligent ECG monitoring; the chapters that follow build on it by reviewing existing approaches, defining the problem and objectives, presenting the proposed system, and evaluating the resulting NAS-optimized model.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews existing research on neural architecture search, lightweight ECG classification, multi-objective optimization, hardware-aware latency reduction, power-efficient edge inference and model quantization. Taken together, the reviewed studies point to a clear need: automated architecture design that can balance classification performance against the resource limits of wearable and embedded platforms.

2.1 NEURAL ARCHITECTURE SEARCH

Neural Architecture Search (NAS) automates the design of neural network architectures within a predefined search space. Rather than hand-picking the number of layers, filters, kernel sizes, strides and other architectural components, NAS treats architecture selection as an optimization problem - a framing that is especially useful for embedded applications, where model accuracy has to be reached under tight memory and computational limits.

Early NAS work showed that automated search could discover effective architectures, though at considerable computational cost [12]. Later approaches focused on cutting this search cost and tailoring the process to specific deployment requirements [8]. ProxylessNAS, for instance, was built to search directly for architectures suited to the target task and hardware, reducing reliance on expensive proxy tasks [10].

For the present ECG application, NAS operates within a constrained one-dimensional convolutional search space, with the architecture free to vary in the number of stages, filters, kernel length, stride, pooling, dropout, dense-layer width and activation function. Restricting the search space this

way keeps unnecessary architectural complexity in check while still letting the search explore a range of lightweight CNN configurations.

2.2 LIGHTWEIGHT DEEP LEARNING FOR ECG CLASSIFICATION

Deep learning has been applied extensively to automatic ECG and arrhythmia classification [2]. CNN-based models can learn morphological features directly from ECG waveforms, but larger networks are often hard to deploy on wearable devices given memory, computation and latency limits [3].

Lightweight ECG classification research has consequently focused on cutting parameter counts while holding on to diagnostic performance [13]. Kim et al. built a lightweight CNN for real-time arrhythmia classification on a low-power wearable ECG device and showed it running on an ARM Cortex-M4 microcontroller [14]. Their reported execution time and current consumption make the point that model design has to weigh computational and electrical cost together, not accuracy in isolation.

This direction continues in more recent work through compact architectures built specifically for resource-constrained ECG systems [15]. EdgeECG, for example, proposes an ultra-lightweight network with a model compression strategy for real-time arrhythmia classification on low-power edge devices. Studies like these reinforce the same point: architectures need to preserve useful ECG classification performance while asking less of the deployment platform.

2.3 MULTI-OBJECTIVE NEURAL ARCHITECTURE SEARCH

Traditional architecture optimization tends to treat classification accuracy as the main objective. But a highly accurate model can carry a large parameter count, demand substantial memory and computation, and

introduce inference delay an embedded platform cannot afford. Multi-objective NAS addresses this by weighing several objectives at once [16].

This project uses macro-F1 as the main classification objective, alongside minimizing model size and inference latency. Macro-F1 suits the ECG dataset well because its class distribution is heavily imbalanced: by computing the F1 score separately for each class and then averaging, macro-F1 gives every class equal weight, keeping the majority class from dominating architecture evaluation.

Multi-objective NAS also opens the door to several Pareto-efficient architectures rather than collapsing everything into a single score [17]. An architecture with higher macro-F1 but greater latency can sit on the Pareto front alongside another that is smaller and faster but slightly less accurate, letting the final choice reflect the requirements of the intended deployment platform.

2.4 LATENCY-AWARE AND HARDWARE-AWARE NAS

Real-time wearable applications need low inference latency, since the model has to keep pace with incoming ECG beats without introducing excessive delay. Parameter count and computational complexity are useful proxies for efficiency, but they do not always track actual execution time, because the same architecture can behave differently across hardware platforms [18].

MnasNet introduced platform-aware NAS by feeding measured real-world latency directly into the optimization process, rather than relying purely on indirect measures such as FLOPs [19]. Its results showed that architectures need to be optimized for the platform they will actually run on, since hardware characteristics shape inference speed. ProxylessNAS

made a similar case for direct hardware-aware optimization, underlining the value of considering the target hardware during NAS [10].

This project follows that same principle by factoring inference latency into architecture evaluation. The reported latency figures, however, come from the host-side development environment rather than the ESP32 itself, so measuring latency on the target device remains an important next step.

2.5 POWER AND ENERGY-AWARE EDGE COMPUTING

Power consumption matters for wearable, battery-powered systems, since continuous sensing and repeated inference can eat into operating lifetime. A smaller model tends to need less computation and memory movement, which can translate into lower energy use [11]. Parameter count or FLOPs alone, though, are not a reliable stand-in for power consumption, because energy use depends heavily on the hardware platform and how the model actually executes on it.

Energy-aware NAS research has begun treating energy as an explicit optimization objective in its own right [20]. Several studies note that latency is an imperfect proxy for energy, since different neural operations carry different energy costs. More recent hardware-aware approaches instead build in device-specific energy estimation or direct on-device profiling, letting accuracy, latency and energy be optimized together [18].

In this project, power is treated as an important deployment concern, but it is not directly measured or used as an explicit objective in the reported NAS experiment; the current optimization is limited to macro-F1, parameter count and inference latency. Measuring ESP32 power directly and building explicit power-aware NAS are left as future extensions.

2.6 QUANTIZATION AND RESOURCE-EFFICIENT DEPLOYMENT

Quantization is a key technique for cutting the resource requirements of neural networks bound for embedded deployment [7]. Full-integer INT8 quantization converts floating-point weights and activations into eight-bit integer form, shrinking memory requirements and making integer-based inference a better fit for microcontroller-class platforms [20].

Here, post-training INT8 quantization is applied after the NAS architecture has been selected and fully trained, on a network with 1,269 trainable parameters. Doing so drops the estimated runtime memory from 43,476 bytes (FP32) to 10,869 bytes (INT8) - roughly a fourfold reduction - while the serialized model size falls from 9.5 kB to 8.23 kB.

The literature also makes clear that architecture optimization and embedded deployment need to be considered together. Lightweight ECG models have been successfully moved onto microcontroller platforms [14], and hardware-aware NAS frameworks stress evaluating architectures against the characteristics of the actual target device [17]. In this project, the immediate deployment step is moving the quantised model from the host or edge server onto the ESP32 using an embedded inference runtime such as TensorFlow Lite Micro [6].

2.7 SUMMARY OF LITERATURE REVIEW

Overall, the literature makes it clear that neural architecture design for wearable ECG systems cannot rest on classification accuracy alone. Lightweight CNN research underlines the importance of smaller models [13], hardware-aware NAS shows that actual latency is tied to the deployment platform [19], and energy-aware studies argue that power

should be optimized as its own objective rather than inferred from parameter count or FLOPs [20].

The reviewed work therefore motivates a resource-aware, multi-objective NAS framework for the proposed ECG system. Here, macro-F1, parameter count and inference latency are folded into the search, with a 120,000-parameter pre-pruning constraint and Pareto-based selection used to filter out unsuitable architectures before expensive evaluation. The selected architecture ends up with only 1,269 parameters and reaches 0.8899 accuracy and 0.6188 macro-F1 after INT8 quantization, while cutting estimated runtime memory substantially relative to the FP32 model.

One gap that remains is the lack of direct target-device latency and power measurements in the current implementation. The next chapter turns to the proposed methodology and experimental framework, walking through the architecture-search procedure, objective evaluation, Pareto analysis, quantization and deployment strategy in detail.

CHAPTER 3

PROBLEM IDENTIFICATION

This chapter pins down the problem the project addresses: deploying neural networks for wearable ECG arrhythmia classification. It focuses on where conventional, manually designed models fall short, and makes the case for multi-objective neural architecture search that balances classification performance against model size, inference latency and, eventually, power.

3.1 PROBLEM STATEMENT

Wearable ECG monitoring needs continuous signal analysis under the tight computational and memory budget of embedded and edge devices. A network optimized purely for classification accuracy can end up with excessive parameters, heavier memory and computation demands, and higher inference cost - all of which make direct deployment on a resource-constrained platform difficult [21].

This problem is sharper still for arrhythmia classification because the ECG dataset is highly imbalanced [22]. Optimizing for overall accuracy alone lets the majority Normal class dominate evaluation, leaving minority classes such as Fusion under-represented. A sound architecture-selection process therefore needs a balanced classification measure like macro-F1, alongside deployment-related resource requirements.

Inference latency is a further constraint for real-time ECG monitoring, since predictions need to be produced promptly as beats are processed. Running on a battery-powered wearable node also makes computational and energy efficiency important design considerations. That said, the present implementation does not directly measure or optimize

classifier power consumption, and the reported latency comes from the host-side development environment rather than the ESP32 itself.

The problem this work sets out to solve, then, is building an automated architecture-selection mechanism that can find compact ECG classification networks while keeping predictive performance balanced and deployment cost low. The proposed approach tackles this through multi-objective NAS using macro-F1, parameter count and inference latency, with power kept in view as an important direction for future hardware-aware optimization.

3.2 LIMITATIONS OF EXISTING SYSTEMS

- Manually designed neural networks can carry unnecessary layers, filters and parameters, since architecture choices are usually driven by classification performance rather than deployment constraints.
- Optimizing for accuracy alone tends to favour increasingly complex models and does not fairly represent performance across highly imbalanced ECG classes.
- Larger models need more weight and activation memory, which becomes a serious constraint when deploying deep-learning models on microcontroller-class wearable platforms [6].
- Parameter count by itself does not measure actual inference speed, since execution latency depends on the characteristics of the target hardware.
- Host-side latency measurements cannot guarantee the same performance on the ESP32 platform, so genuine hardware-aware optimization requires benchmarking on the actual target device.

- A fixed architecture design offers no systematic way to explore different combinations of convolutional filters, kernel sizes, strides, pooling operations and dense-layer dimensions.
- Power consumption is an important concern for battery-powered wearable systems, but the current implementation does not yet provide empirical energy-per-inference or battery-life measurements for the classifier.
- Post-training quantization can shrink deployment memory, but quantization by itself cannot say whether the underlying architecture suits the target device - architecture design and deployment optimization need to be considered together.

3.3 RESEARCH GAP

The reviewed literature points to a gap between conventional, accuracy-oriented network design and what wearable ECG deployment actually requires. Existing approaches offer strong classification capability, lightweight architectures or compression techniques individually, but architecture selection needs to account for balanced ECG classification performance and resource constraints at the same time. This project addresses that gap by framing architecture selection as a multi-objective optimization problem in which macro-F1 is maximized while parameter count and inference latency are minimized.

The proposed NAS framework searches a predefined one-dimensional convolutional architecture space, applying a 120,000-parameter pre-pruning constraint before any expensive candidate evaluation. Evolutionary search - tournament selection, mutation and crossover - then explores the candidate architectures, while Pareto analysis surfaces architectures that represent different performance-resource trade-offs.

This gap also extends to deployment-aware optimization in practice. This study shows the selected architecture can be reduced to 1,269 trainable parameters, and that INT8 quantization cuts estimated runtime memory substantially. Actual inference on the ESP32 and direct power measurement, however, remain to be done - future work will focus on target-hardware benchmarking and folding measured latency and power into the architecture-search objective.

This research problem sets up the next chapter, which lays out the specific objectives of the proposed multi-objective NAS framework. The chapters after that develop the methodology for evolutionary search, Pareto-based architecture selection, quantization and eventual deployment on the target wearable platform.

CHAPTER 4

OBJECTIVE

This chapter sets out the objectives of the proposed multi-objective Neural Architecture Search (NAS) framework for wearable ECG arrhythmia classification, centred on automatically finding a compact, efficient convolutional architecture that keeps classification performance balanced while cutting model size and inference cost for resource-constrained deployment.

4.1 OBJECTIVES OF THE PROJECT

1. To frame the ECG neural-network architecture-selection problem as a multi-objective Neural Architecture Search problem, rather than relying solely on manual architecture design.
2. To maximize balanced ECG classification performance using macro-F1 as the principal predictive objective, ensuring minority arrhythmia classes are weighed alongside the majority Normal class during architecture selection.
3. To minimize the number of trainable parameters in candidate architectures, so the selected model needs less storage and memory and suits constrained wearable and embedded platforms better.
4. To enforce a 120,000-parameter architecture-space constraint, rejecting oversized candidate architectures before expensive proxy training and cutting unnecessary computation during the NAS process.
5. To minimize inference latency during architecture evaluation, so the selected ECG classifier can support responsive real-time monitoring; the current evaluation uses host-side latency

measurements, with direct ESP32 latency measurement planned as a later deployment objective.

6. To employ evolutionary NAS - tournament selection, mutation and single-point crossover - for systematic exploration of the predefined one-dimensional convolutional search space, configured with a population of 10 architectures, four generations and four proxy-training epochs per candidate.
7. To identify Pareto-efficient architectures representing different trade-offs between classification performance and deployment cost, followed by weighted fitness-based selection tailored to the intended resource-efficiency requirement.
8. To apply full-integer INT8 post-training quantization to the selected architecture, reducing its deployment memory footprint while preserving classification performance.
9. To evaluate the selected architecture against the manually designed baseline on classification performance, parameter count, memory requirements and inference latency, and to study the resulting accuracy-efficiency trade-off.
10. To lay the groundwork for future hardware-aware NAS by deploying the quantised model on the ESP32 and folding measured target-device latency and power or energy consumption in as additional optimization objectives; power is treated as a future deployment objective since it is not directly measured in the current experiment.

4.2 SCOPE OF THE PROJECT

This work covers the development and evaluation of a resource-aware, multi-objective NAS framework for wearable ECG arrhythmia classification. It starts with preparing the ECG input representation - the

normalized ECG beat waveform combined with two causal RR-context features, giving a 150×3 input tensor - then uses a predefined one-dimensional convolutional search space to generate candidate architectures by varying parameters such as the number of stages, filters, kernel length, stride, pooling operation, dropout rate, dense-layer width and activation function.

Architecture-space pre-pruning, capped at 120,000 parameters, discards oversized architectures before proxy training. The remaining candidates go through short proxy training runs and are scored on macro-F1, parameter count and inference latency. Evolutionary search runs over a population of 10 architectures across four generations, using tournament selection, mutation and single-point crossover to generate new candidates - 50 candidates in total under the reported search configuration.

The scope also covers Pareto-based analysis of the candidate architectures: the Pareto front identifies architectures that are not simultaneously inferior across the considered objectives, and weighted fitness guides selection according to different accuracy-resource priorities. This lets the search weigh the practical trade-off between balanced classification performance and deployment efficiency, rather than treating accuracy as the only requirement.

Once selected, the network is fully trained and evaluated against the manual baseline. The resulting architecture - two convolutional stages, global average pooling, a 32-unit dense layer and a five-class softmax output - totals 1,269 trainable parameters. The scope further covers full-integer INT8 post-training quantization and a comparison of the FP32 and INT8 variants on model size, estimated runtime memory, classification performance and inference latency.

This phase does not extend to direct measurement of classifier power consumption on the ESP32, and the reported latency values come from the host-side development environment rather than actual microcontroller execution. Measured device-level latency, energy per inference and battery-life analysis therefore sit outside the current experimental scope and are left for future work.

The next step is transferring the quantised model to the ESP32 using an embedded inference runtime such as TensorFlow Lite Micro, and measuring its actual memory, latency and power behaviour on the target platform. From there, a larger search budget, multiple random seeds and explicit power-aware optimization can extend the framework toward more comprehensive hardware-aware, multi-objective NAS.

Taken together, these objectives define the measurable scope of the project: an automatically searched ECG classifier that balances predictive performance with model compactness and inference efficiency, followed by quantization for resource-efficient deployment. The next chapter presents the proposed methodology and system architecture used to implement these objectives and evaluate the resulting NAS-based model.

CHAPTER 5

PROBLEM SOLUTION

This chapter presents the proposed solution to the problem set out in the previous chapter, treating ECG neural-network design as a resource-aware, multi-objective optimization problem. The approach brings together Neural Architecture Search, Pareto-based selection, evolutionary optimization and post-training quantization to arrive at a model that is accurate yet compact enough for wearable ECG applications.

5.1 PROPOSED SYSTEM OVERVIEW

The proposed system automatically searches for a suitable one-dimensional convolutional neural network for wearable ECG arrhythmia classification. Rather than fixing the network structure by hand, the architecture is drawn from a predefined search space covering different numbers of stages, filters, kernel sizes, strides, pooling operations, dropout rates, dense-layer widths and activation functions.

The ECG input combines the normalized beat waveform with two causal RR-context features, producing a 150×3 input tensor. Candidate architectures first go through architecture-space pre-pruning against a 120,000-parameter ceiling, with anything over that limit discarded before proxy training.

Surviving architectures are then judged on three objectives: classification performance via macro-F1, and minimized parameter count and inference latency. Pareto optimization and weighted fitness analyze the candidates next, after which evolutionary mutation and crossover carry the search forward.

The overall solution can be represented as:

$$\begin{aligned}
&\text{ECG Input} \rightarrow \text{Search Space} \rightarrow \text{PrePruning} \rightarrow \quad (5.1) \\
&\text{Proxy Training} \rightarrow \text{MultiObjective Evaluation} \rightarrow \\
&\text{Pareto Selection} \rightarrow \text{Evolutionary Search} \rightarrow \\
&\text{Best Architecture} \rightarrow \text{Full Training} \rightarrow \\
&\text{INT8 Quantization} \rightarrow \text{Deployment}
\end{aligned}$$

The wearable node itself is built around ECG acquisition and embedded monitoring on an ESP32. In the current implementation, the ESP32 handles acquisition, packetisation, display and wireless transmission, while the INT8 classifier runs on the edge-side processing environment.

5.2 WORKING PRINCIPLE

The proposed NAS system starts by generating a population of candidate CNN architectures, each described by a vector of design variables:

$$A_i = \{S, F, K, R, P, D, H, \phi\} \quad (5.2)$$

where S is the number of convolutional stages, F represents the filter configuration, K represents kernel length, R represents stride, P represents pooling selection, D represents dropout, H represents dense-layer width, and ϕ represents the activation function. The activation search space includes ReLU and ReLU6 [23].

The search runs as an evolutionary strategy over a population of 10 architectures across four generations, with each surviving architecture trained for four proxy epochs before evaluation. Tournament selection picks promising candidates, while mutation and single-point crossover generate new ones - 50 candidates in total are reported as evaluated.

The basic search cycle is:

$$A_0 \rightarrow \text{PrePruning} \rightarrow \text{Proxy Training} \rightarrow \text{Evaluation} \rightarrow \text{Selection} \rightarrow \text{Mutation/Crossover} \rightarrow A_{\text{next}} \quad (5.3)$$

The parameter constraint is expressed as:

$$N_{\text{params}}(A_i) \leq 120000 \quad (5.4)$$

where $N_{\text{params}}(A_i)$ is the number of trainable parameters of architecture A_i . Any architecture violating Equation (5.4) is rejected before expensive candidate training.

This keeps the evolutionary process from wasting training effort on architectures that can never satisfy the model-size requirement. Once the search concludes, the selected architecture is fully trained and then subjected to INT8 post-training quantization.

5.3 MULTI-OBJECTIVE OPTIMIZATION MODEL

The central part of the proposed solution is the simultaneous consideration of classification quality and deployment cost. Since the ECG dataset is highly imbalanced, macro-F1 is used instead of raw accuracy as the principal classification objective. For class k , the F1 score is calculated as:

$$F1_k = \frac{2P_kR_k}{P_k + R_k} \quad (5.5)$$

where P_k is the precision and R_k is the recall for class k .

For K classes, the macro-F1 score is:

$$F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{2P_kR_k}{P_k + R_k} \quad (5.6)$$

Weighting the classes equally this way keeps the majority Normal class from dominating the architecture search outright.

The multi-objective problem can accordingly be written conceptually as:

$$\max F1_{macro}(A) \quad (5.7)$$

$$\min N_{params}(A) \quad (5.8)$$

$$\min L(A) \quad (5.9)$$

where $L(A)$ denotes inference latency.

The three objectives can be represented together as:

$$f(A) = [-, F, 1_{macro}(A)N_{params}(A)L(A)] \quad (5.10)$$

where the negative sign turns the maximization objective into a minimization form, as is standard for multi-objective optimization.

A normalized weighted fitness measure can then be written conceptually as:

$$F(A) = w_1\hat{F}_{macro}(A) - w_2\hat{N}_{params}(A) - w_3\hat{L}(A) \quad (5.11)$$

where w_1, w_2, w_3 represent objective weights and the hat notation denotes normalized objective values. The chapter uses different weighting configurations to represent different levels of accuracy and resource preference.

The key principle at work is that a larger model may classify better but also costs more in memory and computation. The optimization therefore looks for an appropriate balance rather than simply maximizing network size.

5.4 PARETO-BASED ARCHITECTURE SELECTION

Since these objectives can pull in different directions, there may be no single architecture that simultaneously achieves the highest macro-F1, the lowest parameter count and the lowest latency. Pareto optimization is used to surface architectures representing different trade-offs among them.

For two candidate architectures A and B , architecture A dominates B when:

$$F1_{macro}(A) \geq F1_{macro}(B) \quad (5.12)$$

$$N_{params}(A) \leq N_{params}(B) \quad (5.13)$$

$$L(A) \leq L(B) \quad (5.14)$$

and at least one of these inequalities is strict.

An architecture counts as Pareto-efficient when no other candidate dominates it, and the set of such architectures forms the Pareto front:

$$\mathcal{P} = \{A_i \mid A_i \text{ is not dominated by any } A_j\} \quad (5.15)$$

Keeping this Pareto front stops the search from collapsing every objective into a single measure too early. Once Pareto analysis is done, a weighted fitness measure guides selection toward the preferred balance of classification performance and resource efficiency.

Two weighting configurations are used in this work: an edge-balanced configuration that weighs resource efficiency more heavily, and an accuracy-first configuration that emphasizes predictive performance instead. Between them, the search can reflect different deployment requirements without assuming any one architecture suits every case.

5.5 LATENCY, POWER AND RESOURCE-EFFICIENCY MODEL

Inference latency here is the time the model needs to process an input and produce a prediction. Lower latency is preferable for real-time wearable ECG monitoring, since beats arrive continuously and predictions need to keep up.

The resource-aware design also considers how model complexity relates to deployment cost. A simplified conceptual relationship can be written as:

$$\begin{aligned} \text{Model Complexity} \uparrow & \quad (5.16) \\ \Rightarrow \text{Parameters, Memory, Computation and Latency} \end{aligned}$$

while:

$$\text{Model Complexity} \downarrow \Rightarrow \text{Resource Requirement} \downarrow \quad (5.17)$$

provided the reduction does not come at too steep a cost to classification performance.

The present experiment reports host-side latency values rather than latency measured directly on the ESP32, so these figures should not be read as a direct measurement of final wearable-device performance. Measured target-device latency is flagged as an important requirement for future hardware-aware NAS work.

Power consumption matters too, for a battery-operated wearable system. A smaller model can potentially cut computation and memory movement and, with it, energy consumption - but this work does not measure energy per inference, battery lifetime or ESP32 power draw directly. Power is therefore not treated as an experimentally optimized objective in the current implementation.

A future power-aware formulation can be expressed as:

$$\begin{aligned}
& \max F 1_{macro}(A) \\
& \min L_{device}(A) \\
& \min E_{inf}(A)
\end{aligned} \tag{5.18}$$

where L_{device} is measured target-device latency and E_{inf} is energy consumed per inference.

Energy consumed per inference can be represented by:

$$E_{inf} = P_{avg} \times t_{inf} \tag{5.19}$$

where P_{avg} is average power consumption and t_{inf} is inference time. This represents a future hardware-aware extension rather than an experimentally measured result of the present work.

5.6 SELECTED ARCHITECTURE AND QUANTIZATION

The evolutionary search settled on a compact convolutional architecture built from two convolutional stages followed by a classification head. The first stage uses 16 filters with a kernel size of 7, stride 2, ReLU6 activation and dropout of 0.1; the second uses 8 filters with a kernel size of 3, stride 2, ReLU6 activation, max pooling and dropout of 0.1.

The classification head itself consists of global average pooling followed by a 32-unit dense layer and a five-unit softmax output, giving an architecture with only 1,269 trainable parameters. Global average pooling here avoids the parameter growth that flattening large feature maps before the dense layer would otherwise cause.

The selected architecture can be represented as:

$$\begin{aligned}
150 \times 3 \rightarrow \text{Conv}(16, 7, 2) \rightarrow \text{Conv}(8, 3, 2) \rightarrow \text{GAP} \\
\rightarrow \text{Dense}(32) \rightarrow \text{Softmax}(5)
\end{aligned} \tag{5.20}$$

where GAP denotes global average pooling.

Once training is complete, post-training full-integer INT8 quantization is applied, converting floating-point weights and activations into eight-bit integer representations using representative calibration data.

The approximate storage and memory relationship before and after Quantization is:

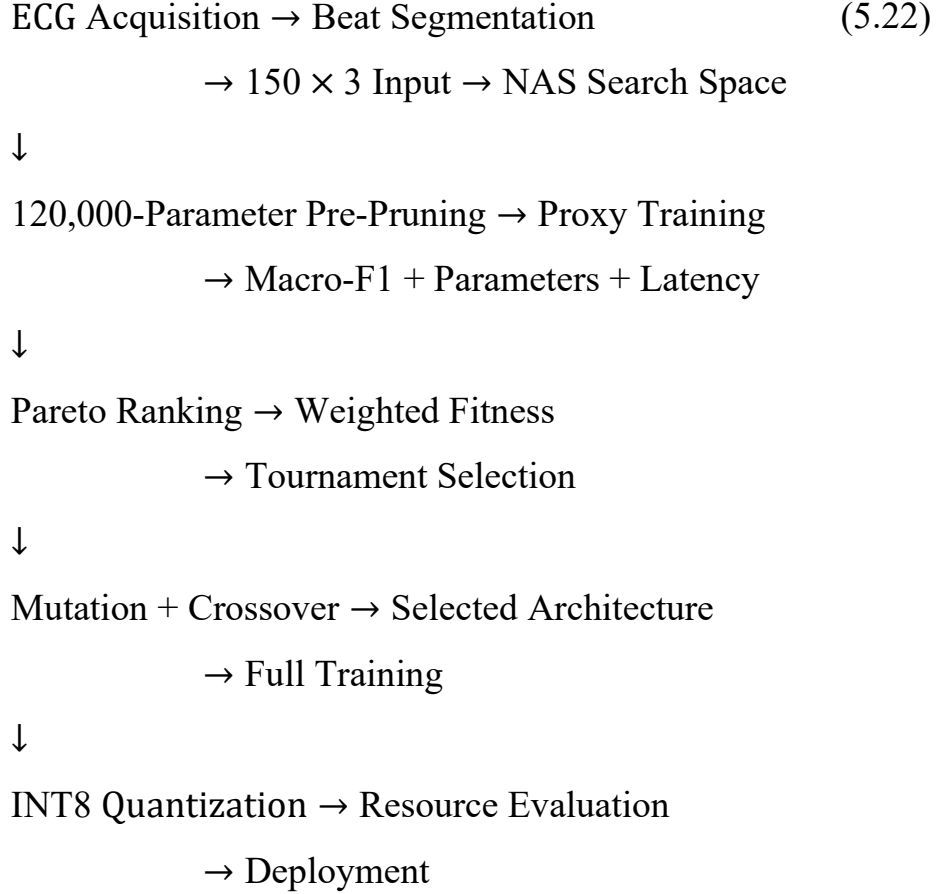
$$\begin{aligned} M_{FP32} &\approx 4N \\ M_{INT8} &\approx N \end{aligned} \tag{5.21}$$

where N represents the number of stored numerical values, ignoring additional implementation overhead.

The INT8 model consequently reduces estimated runtime memory from 43,476 bytes to 10,869 bytes - roughly a fourfold cut. The slightly higher INT8 latency seen on the development host is not a general drawback of INT8; the host and the target embedded hardware simply execute differently.

5.7 SYSTEM FLOW AND SOLUTION SUMMARY

The complete solution brings signal preparation, architecture search, multi-objective evaluation and deployment optimization together into a single workflow:



The proposed solution shows the practical value of resource-aware NAS in concrete terms: a 1,269-parameter architecture against a 13,973-parameter manually designed baseline. The final INT8 model reaches 0.8899 accuracy and 0.6188 macro-F1, with estimated runtime memory well below that of the corresponding FP32 model.

This solution is deliberately framed as a deployment-oriented architecture-selection methodology, not a claim that the selected architecture is globally optimal. The search evaluated 50 candidates over four generations from a single random seed, and the current experiment does not directly measure ESP32 inference latency or classifier power consumption.

This chapter has laid out the full solution framework for balancing ECG classification performance with model complexity and inference efficiency through multi-objective NAS. The next chapter turns to the implementation methodology - dataset preparation, NAS configuration, training procedure, evaluation metrics and experimental setup - used to validate this solution.

CHAPTER 6

BLOCK DIAGRAM

This chapter presents the block diagram of the proposed multi-objective NAS system for wearable ECG arrhythmia classification, tracing the complete flow from ECG acquisition and preprocessing through NAS-based architecture selection, model optimization, quantization and deployment, and showing how the major hardware and software components fit together.

6.1 OVERALL SYSTEM ARCHITECTURE

The proposed architecture is organized into six functional units: ECG sensing and acquisition, signal preprocessing and feature preparation, multi-objective NAS, model training and evaluation, quantization and deployment, and monitoring and analysis. Figure. 6.1 shows the overall architecture of the proposed wearable ECG arrhythmia classification system.

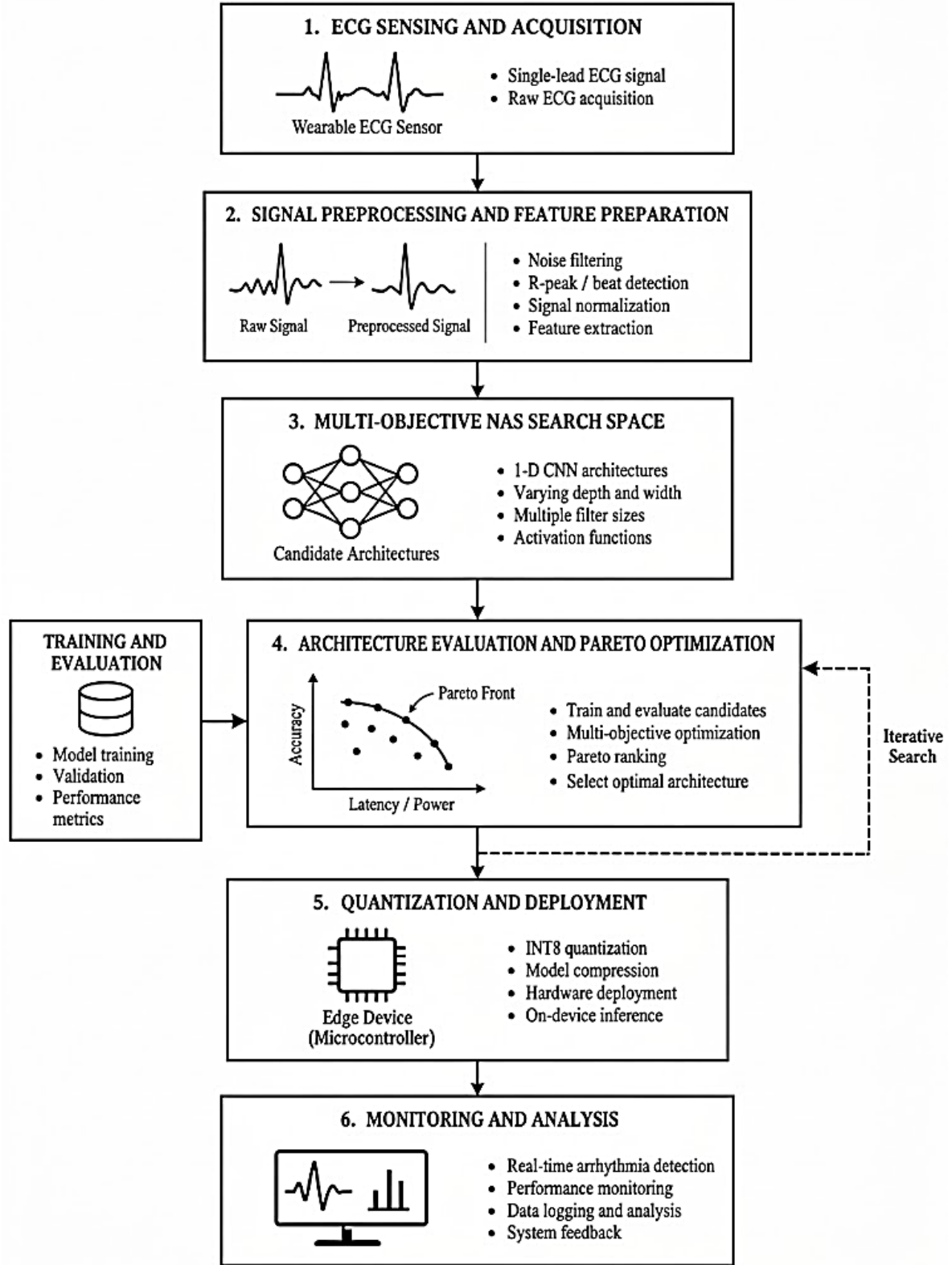


Figure. 6.1 Overall system architecture of the proposed multi-objective NAS-based wearable ECG arrhythmia classification system

The ECG sensing and acquisition unit pulls the single-lead ECG signal from the sensing hardware and passes it on for processing. From there, the signal preprocessing and feature preparation unit segments beats and builds the model input from the normalized ECG waveform plus two causal RR-context features, giving a 150×3 input representation.

The multi-objective NAS unit generates candidate one-dimensional CNN architectures from a predefined search space, with candidates varying in the number of convolutional stages, filter count, kernel length, stride, pooling, dropout, dense-layer width and activation function; ReLU and ReLU6 make up the activation search space.

The model training and evaluation unit applies the 120,000-parameter pre-pruning constraint, runs proxy training and scores the surviving candidates on macro-F1, parameter count and inference latency, before Pareto analysis and weighted fitness guide the evolutionary selection process.

The quantization and deployment unit takes the selected architecture after full training and applies full-integer INT8 post-training quantization, then evaluates the resulting model for memory and inference characteristics ahead of deployment. In the reported implementation, the quantised model stays on the edge-side processing environment, while the ESP32 handles ECG acquisition, packetisation, display and wireless transmission.

The monitoring and analysis unit presents classification results together with resource-related measurements for evaluating the selected architecture. The complete architecture is built so that model selection weighs predictive performance and deployment requirements together, rather than treating classification accuracy as the only design criterion.

6.2 DESCRIPTION OF FUNCTIONAL UNITS

1. ECG Sensing and Acquisition Unit – pulls the ECG waveform from the wearable sensing hardware and supplies the raw signal needed for beat-level classification.
2. Signal Preprocessing and Feature Preparation Unit – handles signal preparation and beat segmentation, building the 150×3 input from the normalized ECG beat waveform and two causal RR-context features.
3. Multi-Objective NAS Unit – generates candidate one-dimensional CNN architectures and explores different configurations within the predefined search space.
4. Architecture Evaluation and Selection Unit – applies the 120,000-parameter pre-pruning limit, runs proxy training, and evaluates candidates on macro-F1, parameter count and inference latency, using Pareto-based analysis followed by weighted fitness for final selection.
5. Quantization and Deployment Unit – fully trains the selected architecture and converts the floating-point model into a full-integer INT8 model to cut deployment resource requirements; the reported INT8 model holds 1,269 parameters with an estimated runtime memory requirement of 10,869 bytes.
6. Monitoring and Analysis Unit – records and reviews classification performance, model size, memory requirement and inference latency to judge the practical behaviour of the selected architecture.

Although described separately, the six units function as a single pipeline. The sensing unit feeds ECG data to the preprocessing unit, which prepares the representation the architecture search and classifier need. The

NAS unit generates and evaluates alternative architectures, and the selection unit filters them against the defined performance and resource objectives, before the chosen network moves on to full training, quantization and deployment.

The architecture also keeps architecture-level optimization separate from model-level optimization: NAS decides the network structure, while quantization shrinks the deployment footprint once that structure is fixed. This separation lets the search focus purely on architectural trade-offs while still gaining the benefits of post-training model compression [11].

One notable feature of the architecture is its distinction between the current implementation and future deployment. Today, latency is evaluated on the host-side environment rather than the ESP32, and power consumption is not directly measured - leaving the architecture as a foundation for future hardware-aware optimization once measured target-device latency and power can be worked into the search.

6.3 SYSTEM FLOW

The system flow starts with ECG acquisition and preparation of the beat-level input: the normalized waveform and two causal RR-context features combine into the 150×3 input representation, and candidate CNN architectures are generated within the predefined NAS search space.

The architecture search begins by applying the 120,000-parameter pre-pruning condition, removing any candidates over the limit before proxy training. The remaining candidates are trained for a limited number of proxy epochs and evaluated on macro-F1, parameter count and inference latency, as shown in Figure. 6.2.

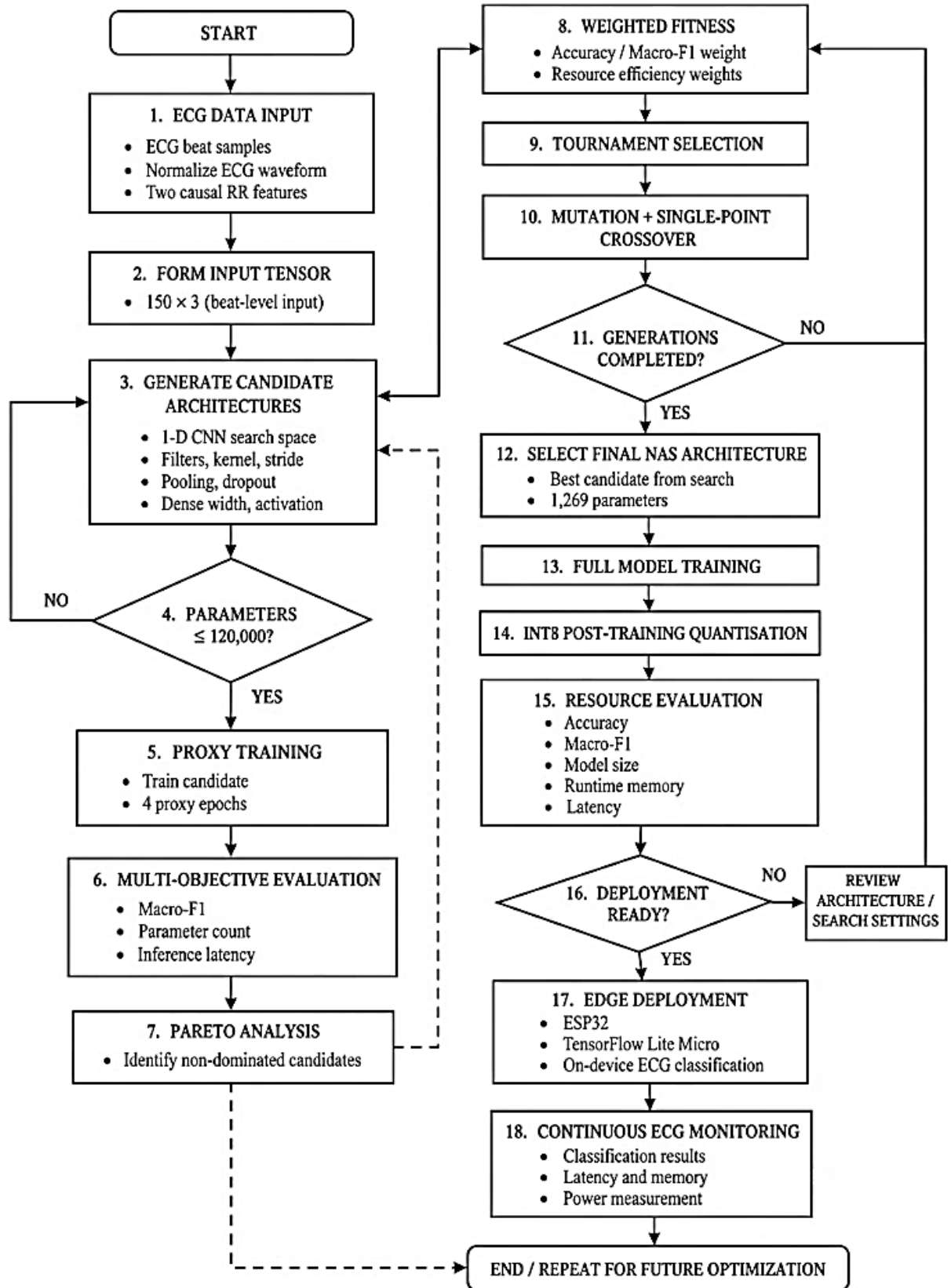


Figure. 6.2 System flow algorithm proposed for the multi-objective NAS-based wearable ECG classification system

As Figure. 6.2 illustrates, the flow runs from ECG acquisition through preprocessing, candidate generation, pre-pruning, proxy training and multi-objective evaluation. Pareto analysis then identifies architectures offering useful trade-offs, after which weighted fitness and evolutionary selection take over; mutation and single-point crossover generate new candidates, and the search continues across the defined generations.

Once the search finishes, the selected architecture is fully trained on the ECG dataset and then converted into a full-integer INT8 representation, before being re-evaluated for classification performance and deployment resources. The final architecture in this study - two convolutional stages, global average pooling, a 32-unit dense layer and a five-class softmax output - holds 1,269 trainable parameters.

This flow also underlines the gap between architecture selection and deployment measurement: the current implementation establishes resource characteristics through host-side evaluation, with execution on the ESP32 itself as the next deployment step. Moving the quantised model to the ESP32 through an embedded runtime such as TensorFlow Lite Micro will make it possible to gather actual target-device memory and latency measurements.

The complete processing sequence can be summarized as: ECG acquisition, beat segmentation, 150×3 input formation, NAS search, parameter pre-pruning, proxy training, macro-F1 and resource evaluation, Pareto ranking, weighted fitness selection, evolutionary mutation and

crossover, final architecture selection, full training, INT8 quantization and deployment.

6.4 NAS ARCHITECTURE

The NAS architecture is the central optimization component of the proposed system. Instead of a manually fixed CNN structure, the search explores a constrained family of one-dimensional convolutional networks, with architecture variables covering the number of stages, filter count, kernel length, stride, pooling, dropout, dense-head width and activation function.

The evolutionary search runs a population of 10 candidate architectures over four generations, with four proxy-training epochs per candidate. Tournament selection drives reproduction, and mutation with single-point crossover produces new candidate architectures - 50 candidates are reported as evaluated in total.

The selected NAS architecture is:

- Input: 150×3
- Conv Stage 1: 16 filters, kernel size 7, stride 2, ReLU6, dropout 0.1
- Conv Stage 2: 8 filters, kernel size 3, stride 2, ReLU6, max pooling, dropout 0.1
- Global Average Pooling
- Dense layer: 32 units
- Output layer: 5-unit softmax
- Total trainable parameters: 1,269

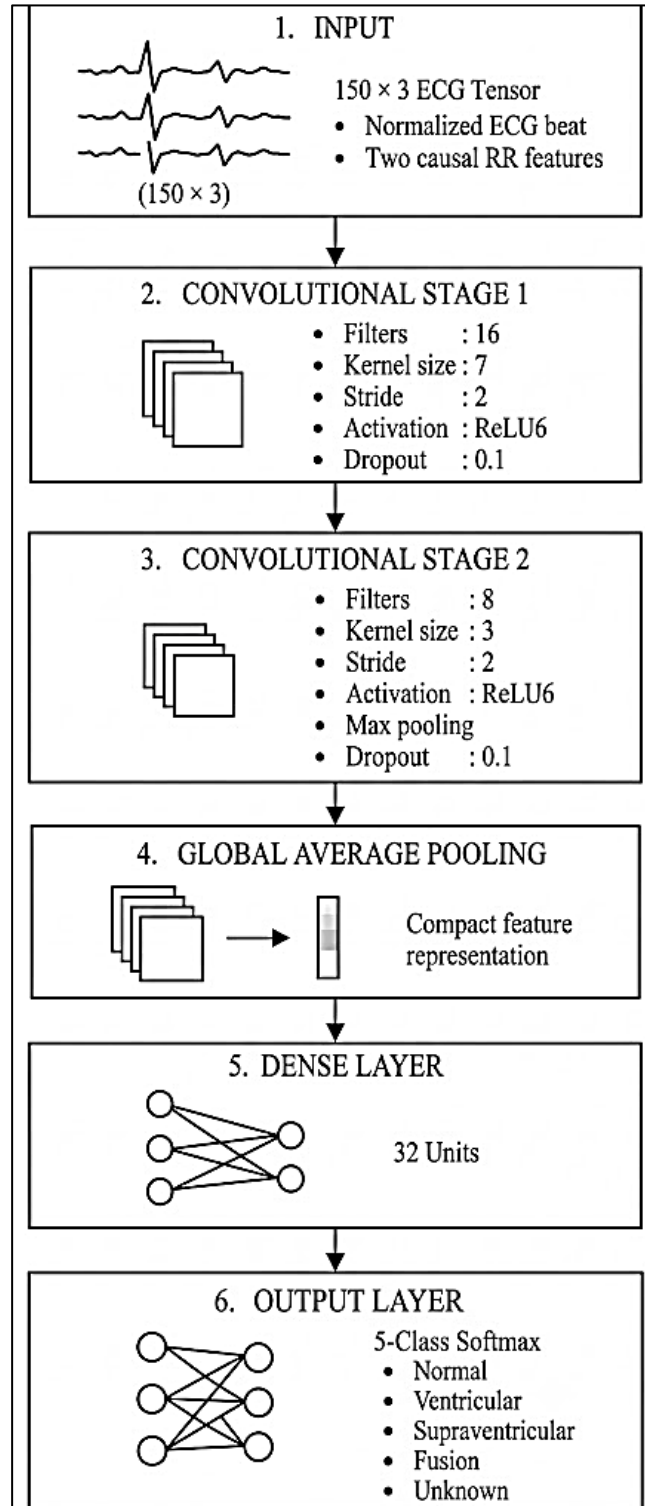


Figure. 6.3 Selected NAS architecture for wearable ECG arrhythmia classification

As Figure. 6.3 shows, the selected architecture takes the 150×3 ECG representation through two compact convolutional stages before

global average pooling and final classification. Reducing the filter count from 16 to 8 and using global average pooling both help keep parameter growth in check and the classification head compact.

This architecture is subsequently quantised through full-integer INT8 post-training quantization. The reported INT8 model keeps the same parameter count while cutting estimated runtime memory from 43,476 bytes to 10,869 bytes, and reaches 0.8899 accuracy and 0.6188 macro-F1 on the reported held-out evaluation.

CHAPTER 7

METHODOLOGY

This chapter walks through the methodology used to develop and evaluate the proposed multi-objective NAS framework for wearable ECG arrhythmia classification - ECG input preparation, constrained architecture generation, evolutionary search, multi-objective evaluation, Pareto-based selection, quantization and deployment-oriented performance assessment.

7.1 ECG DATA PREPARATION AND INPUT REPRESENTATION

The methodology starts with preparing the ECG signals for beat-level classification: the acquired data is processed into individual heartbeat segments, and waveform values are normalized before reaching the network. Two causal RR-context features are added alongside the ECG waveform to supply limited rhythm-related information without drawing on future observations.

The final input representation is a 150×3 tensor combining the normalized ECG beat waveform with the two causal RR-context features, giving the NAS-generated classifier both morphological information from the waveform and contextual information about beat timing.

This preprocessing stage is applied consistently across all candidate architectures, so comparisons between them reflect network design rather than differences in input preparation. The same prepared representation carries through proxy training, final training and evaluation.

7.2 NAS SEARCH SPACE

The proposed NAS framework does not search unrestricted neural-network structures. Instead, a predefined one-dimensional convolutional

search space keeps the possible architectures within configurations suited to ECG beat classification and resource-constrained deployment.

The main architectural variables are the number of convolutional stages, number of filters, kernel length, stride, use of max pooling, dropout rate, width of the dense classification head, and activation function; the activation search space contains ReLU and ReLU6. The final selected architecture uses ReLU6, whose bounded activation range also suits the integer quantization applied later [24].

The parameters considered in the NAS search are summarized below.

Table 7.1 NAS Search-Space Parameters

Parameter	Description
Number of Stages	Number of convolutional processing stages
Number of Filters	Feature maps used in each convolutional stage
Kernel Length	Temporal size of the convolution filter
Stride	Step size used during convolution
Max Pooling	Optional downsampling operation
Dropout Rate	Regularization parameter
Dense Width	Number of units in the classification head
Activation	ReLU or ReLU6

As Table 7.1 shows, the search space is restricted to the architectural parameters that most directly shape feature extraction, model complexity and deployment efficiency - cutting down unnecessary search combinations while keeping enough flexibility for the evolutionary optimization process.

7.3 EVOLUTIONARY NAS SEARCH

The architecture search runs as an evolutionary optimization strategy. An initial population of candidate architectures is drawn from the predefined search space, and every candidate is checked against the maximum parameter constraint before it reaches expensive proxy training.

Architectures above 120,000 trainable parameters are rejected at this architecture-space pre-pruning stage, so computational resources are not spent on candidate networks already outside the permitted model-size range.

The main NAS configuration used in the project is given below.

Table 7.2 Evolutionary NAS Configuration

Search Parameter	Configuration
Search Strategy	Evolutionary NAS
Selection Method	Tournament Selection
Population Size	10
Number of Generations	4
Proxy Training	4 epochs per candidate
Parameter Constraint	120,000
Total Candidates Evaluated	50
Random Seed	1337

As Table 7.2 indicates, the search runs over a population of 10 architectures across four generations. Tournament selection picks out promising candidates, while mutation and single-point crossover produce new architectures - the reported search evaluates 50 candidates in total.

The evolutionary cycle runs through candidate generation, parameter checking, proxy training, objective evaluation, selection, mutation and crossover, repeating until the defined generation count is reached.

7.4 MULTI-OBJECTIVE EVALUATION

Each surviving architecture is judged on its classification performance and deployment-related resource requirements. Macro-F1, rather than raw accuracy, is the principal classification metric here, since the ECG dataset has a highly imbalanced distribution of heartbeat classes and macro-F1 gives a more balanced picture of performance across all of them.

For each heartbeat class, precision and recall combine into a class-level F1 score, and these scores are then averaged across all classes to get macro-F1. This gives every class equal weight during architecture evaluation and stops the majority Normal class from single-handedly determining how good an architecture looks.

Parameter count is the second evaluation objective. Fewer parameters mean less storage for the trained model and, often, lower memory and computational demand at deployment - a point that matters particularly for wearable ECG systems, where memory and processing resources are limited.

Inference latency is the third objective, capturing how long the architecture takes to process an ECG input and produce a classification result. Real-time wearable monitoring needs classification results with minimal delay, so lower-latency architectures are preferred from a deployment standpoint.

All three objectives are considered together during evaluation: macro-F1 is maximized to keep classification performance balanced across heartbeat classes, while parameter count and inference latency are both minimized to reduce model size, memory requirements, and improve responsiveness during real-time operation.

Macro-F1, parameter count and inference latency values are used together when comparing architectures, so the multi-objective evaluation weighs classification quality and deployment efficiency together rather than leaning on accuracy alone.

7.5 PARETO ANALYSIS AND WEIGHTED FITNESS

Because these objectives can conflict, the search may turn up architectures with quite different performance-resource profiles - one might reach higher macro-F1 at the cost of more parameters and greater latency, while another trades a little classification performance for being smaller and faster.

Pareto analysis is used here to identify non-dominated architectures: a candidate is dominated when another architecture matches or beats it on every objective and strictly beats it on at least one, and the architectures that are not dominated by anything form the Pareto front.

This Pareto stage keeps architectures with different trade-offs in play, rather than forcing the search to commit to a single solution too early. After Pareto analysis, a weighted fitness measure guides selection toward the preferred balance between classification performance and resource efficiency.

The reported methodology considers two weighting configurations: an edge-balanced configuration that gives resource efficiency more relative weight, and an accuracy-first configuration that leans more toward predictive performance. Together they show how the preferred final architecture can shift depending on deployment requirements.

This weighted evaluation acts as a decision mechanism once the candidate architectures have already been scored against their individual objectives.

7.6 SELECTED ARCHITECTURE AND FULL TRAINING

Once the evolutionary search finishes, the selected architecture is fully trained on the prepared ECG dataset. The architecture the search identified consists of two convolutional stages followed by global average pooling and a compact classification head.

The first convolutional stage uses 16 filters, a kernel size of 7, stride of 2, ReLU6 activation and dropout of 0.1; the second uses 8 filters, a kernel size of 3, stride of 2, ReLU6 activation, max pooling and dropout of 0.1. The classification head itself consists of global average pooling, a 32-unit dense layer and a five-class softmax output.

The selected architecture is summarized below.

Table 7.3 Selected NAS Architecture

Network Stage	Configuration
Input	150×3 ECG tensor
Convolutional Stage 1	16 filters, kernel 7, stride 2, ReLU6, dropout 0.1
Convolutional Stage 2	8 filters, kernel 3, stride 2, ReLU6, max pooling, dropout 0.1
Convolutional Stage 2	8 filters, kernel 3, stride 2, ReLU6, max pooling, dropout 0.1
Global Feature Reduction	Global Average Pooling
Dense Layer	32 units
Output Layer	5-class softmax
Trainable Parameters	1,269

As Table 7.3 shows, the final architecture is highly compact, with only 1,269 trainable parameters. Global average pooling keeps the classification head from growing unnecessarily, and reducing filters from 16 to 8 across stages contributes further to the compact design [25].

The fully trained NAS model is then compared against the manually designed baseline to see whether this resource reduction comes without an unacceptable drop in classification performance.

7.7 INT8 QUANTIZATION

After full training, the selected floating-point model is converted into a full-integer INT8 model through post-training quantization, turning the floating-point weights and activations into signed eight-bit integer representations using representative calibration data from the training partition [7].

Quantization is applied only after NAS has already selected the architecture, so pre-pruning and quantization serve different purposes: pre-pruning removes oversized architectures during the search itself, while quantization reduces the deployment footprint of whichever architecture the search settles on.

The quantised model is then compared with its FP32 counterpart on model size, estimated runtime memory, classification performance and inference latency, isolating the effect of quantization on deployment efficiency without touching the underlying architecture.

7.8 PERFORMANCE AND RESOURCE EVALUATION

The final evaluation compares the manually designed baseline against the NAS-selected FP32 and INT8 models. The classification results are presented below.

Table 7.4 Classification Performance Comparison

Model	Parameters	Accuracy	Macro-F1
Manual Baseline FP32	13,973	0.8669	0.6203
NAS-selected FP32	1,269	0.8838	0.6148
NAS-selected INT8	1,269	0.8899	0.6188

As Table 7.4 shows, the NAS-selected architecture uses substantially fewer parameters than the manual baseline while matching its balanced classification performance. The INT8 version reaches 0.8899 accuracy and 0.6188 macro-F1, cutting the parameter count by roughly eleven times relative to the manually designed baseline.

The resource measurements for the selected architecture are given below.

Table 7.5 Memory and Latency Comparison of NAS Model Variants

Metric	NAS FP32	NAS INT8
Parameters	1,269	1,269
Serialized Size	9.5 kB	8.23 kB
Estimated Runtime Memory	43,476 B	10,869 B
Host-side Latency	0.0222 ms	0.0351 ms

As Table 7.5 presents, INT8 quantization cuts estimated runtime memory from 43,476 bytes to 10,869 bytes - roughly a fourfold reduction - and the serialized model size from 9.5 kB to 8.23 kB. The INT8 model runs slightly slower on the development host, but this is not treated as a general INT8 drawback, since the host environment and the intended embedded hardware execute quite differently.

The evaluation therefore weighs both predictive and resource-related characteristics together: accuracy and macro-F1 speak to classification capability, while parameter count, model size, runtime memory and latency speak to the practical deployment requirements of the architecture.

7.9 DEPLOYMENT WORKFLOW

The final methodology stage prepares the selected, quantised model for edge deployment. In the current implementation, inference runs in the

host or edge-side environment, while the ESP32 handles ECG acquisition, packetisation, display and wireless transmission.

The complete deployment-oriented workflow is:

1. ECG signal acquisition and beat segmentation.
2. Preparation of the 150×3 input representation.
3. Generation of candidate CNN architectures.
4. Architecture-space pre-pruning using the 120,000-parameter constraint.
5. Proxy training and multi-objective evaluation.
6. Pareto analysis and weighted fitness-based selection.
7. Evolutionary mutation and crossover.
8. Selection of the final NAS architecture.
9. Full training of the selected network.
10. INT8 post-training Quantization.
11. Evaluation of classification and resource characteristics.
12. Preparation for target-device deployment.

The methodology is deliberately structured so architecture optimization happens before final deployment optimization, cutting the risk of settling on a high-performing architecture that later proves unsuitable for the available hardware.

The next step for the implementation is direct deployment of the quantised classifier on the ESP32 using an embedded runtime such as TensorFlow Lite Micro, which would allow actual target-device latency and memory behaviour to be measured. A further extension could fold measured power or energy consumption into the NAS objective, since power is not directly measured in the current implementation.

This chapter has laid out the complete methodology used to turn ECG data into a compact, NAS-selected classifier through constrained evolutionary search, multi-objective evaluation, Pareto analysis and INT8 quantization. The next chapter presents the experimental results and discusses the classification performance, model compactness, memory utilization, latency and deployment-related observations that follow from this methodology.

CHAPTER 8

RESULTS AND DISCUSSION

This chapter presents the experimental results from the proposed multi-objective NAS framework for wearable ECG arrhythmia classification, evaluating the searched architecture on classification performance, model compactness, runtime memory, inference latency and the effect of INT8 quantization, while also flagging the limits of the current host-side evaluation.

8.1 EXPERIMENTAL SETUP

These experiments set out to test whether multi-objective NAS could find a compact ECG classification architecture without giving up useful classification performance, comparing a manually designed FP32 baseline against the architecture the evolutionary NAS process produced and its INT8-quantised version.

The NAS search ran over a predefined one-dimensional convolutional search space with a 120,000-parameter ceiling. The evolutionary configuration used a population of 10 architectures, four generations and four proxy-training epochs per candidate, yielding 50 evaluated candidates in total, with tournament selection, mutation and single-point crossover used throughout.

Architecture evaluation weighed macro-F1, parameter count and inference latency, with macro-F1 chosen over raw accuracy because the ECG classes are highly imbalanced and minority classes need to stay influential during selection.

8.2 NAS SEARCH AND ARCHITECTURE SELECTION

The evolutionary search successfully found a compact architecture within the defined parameter constraint. Architecture-space pre-pruning removed oversized candidates before proxy training, cutting the computation spent on unsuitable architectures.

The selected architecture consists of two one-dimensional convolutional stages followed by global average pooling, a dense layer and a five-class softmax output. The first stage uses 16 filters with a kernel size of 7 and stride of 2; the second uses 8 filters with a kernel size of 3 and stride of 2. Both stages use ReLU6 activation, with max pooling in the second.

As Table 7.3 shows, the final architecture holds only 1,269 trainable parameters. Choosing global average pooling over a large flattened representation helps keep the dense classification head from growing unnecessarily.

8.3 CLASSIFICATION PERFORMANCE

As Table 7.4 presents, the NAS-selected FP32 architecture lifts reported accuracy from 0.8669 for the manual baseline to 0.8838, while cutting the parameter count from 13,973 to 1,269. INT8 quantization pushes accuracy further, to 0.8899, with macro-F1 at 0.6188.

These results show that a substantial cut in architecture size can be achieved without a comparable drop in overall classification capability - the NAS-selected network is roughly eleven times smaller than the manual baseline in trainable parameters.

The macro-F1 values also illustrate why accuracy alone is not enough to judge the ECG classifier: the INT8 model beats the baseline on accuracy, yet its macro-F1 sits slightly below the baseline value, meaning

gains in overall accuracy do not automatically carry over to every heartbeat class.

8.4 CLASS-WISE PERFORMANCE ANALYSIS

Table 8.3 presents the class-wise performance of the final INT8 model, reporting precision, recall and F1-score for each of the five heartbeat categories.

Table 8.1 Class-wise Performance of NAS INT8 Model

Class	Precision	Recall	F1-Score
Normal	0.977	0.903	0.938
Ventricular	0.807	0.934	0.867
Supraventricular	0.487	0.546	0.514
Fusion	0.081	0.342	0.130
Unknown	0.478	0.994	0.645

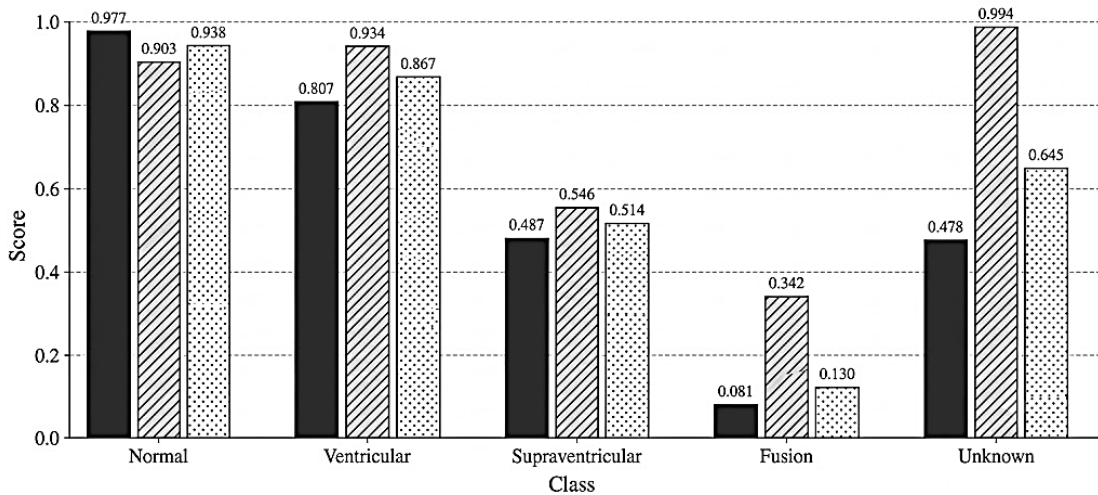


Figure. 8.1 Class-wise Performance Analysis of NAS

As Table 8.1 and Figure. 8.1 show, the Normal and Ventricular classes reach considerably stronger F1-scores than the minority classes. Fusion records the lowest F1-score, at 0.130, while Supraventricular reaches 0.514. Fusion also has particularly limited support in the test set - only 398 beats - which contributes to how unstable minority-class performance is.

This backs up the earlier decision to use macro-F1 during architecture search: a model could post high overall accuracy just by nailing the majority class while still doing poorly on rare arrhythmia categories, whereas macro-F1 gives every class equal weight and a fairer view of performance.

8.5 MEMORY AND MODEL SIZE ANALYSIS

Table 8.2 Memory and Model Size Comparison

Metric	NAS FP32	NAS INT8
Parameters	1,269	1,269
Serialized Size	9.5 kB	8.23 kB
Estimated Runtime Memory	43,476 B	10,869 B

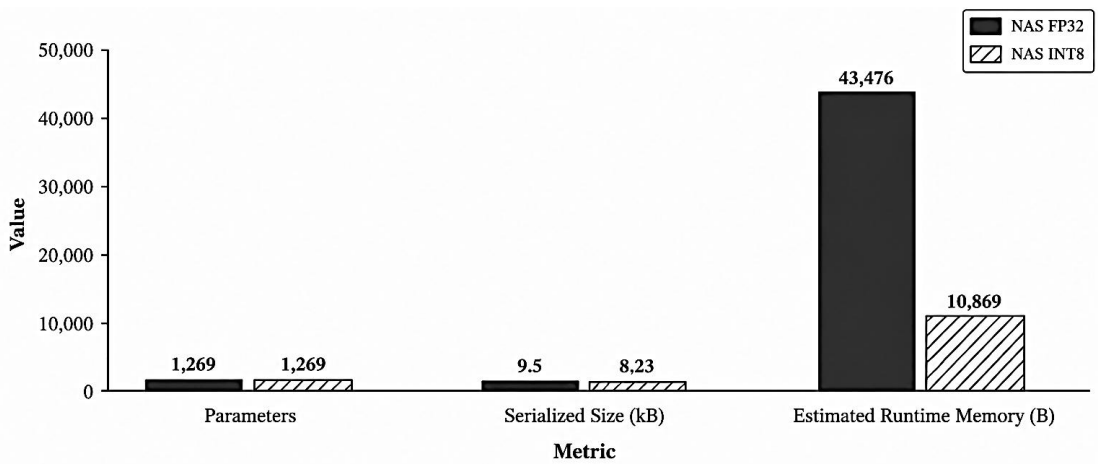


Figure. 8.2 Model Size with Memory Comparison

As Table 8.2 and Figure. 8.2 indicate, INT8 quantization leaves the trainable-parameter count unchanged but substantially cuts estimated runtime memory - from 43,476 bytes (FP32) to 10,869 bytes (INT8), roughly a fourfold reduction, with serialized model size falling from 9.5 kB to 8.23 kB.

This memory reduction matters for wearable and microcontroller-oriented systems, where the model has to share limited memory with the embedded application, communication routines and intermediate

activation storage. These figures confirm that the selected architecture is considerably more compact than the manually designed baseline.

8.6 INFERENCE LATENCY ANALYSIS

Inference latency was one of the resource-related objectives during architecture evaluation, since ECG monitoring needs timely processing of incoming heartbeat samples. Table 8.3 presents the measured latency values for the selected architecture.

Table 8.3 Inference Latency Comparison

Model	Host-side Latency
NAS FP32	0.0222 ms
NAS INT8	0.0351 ms

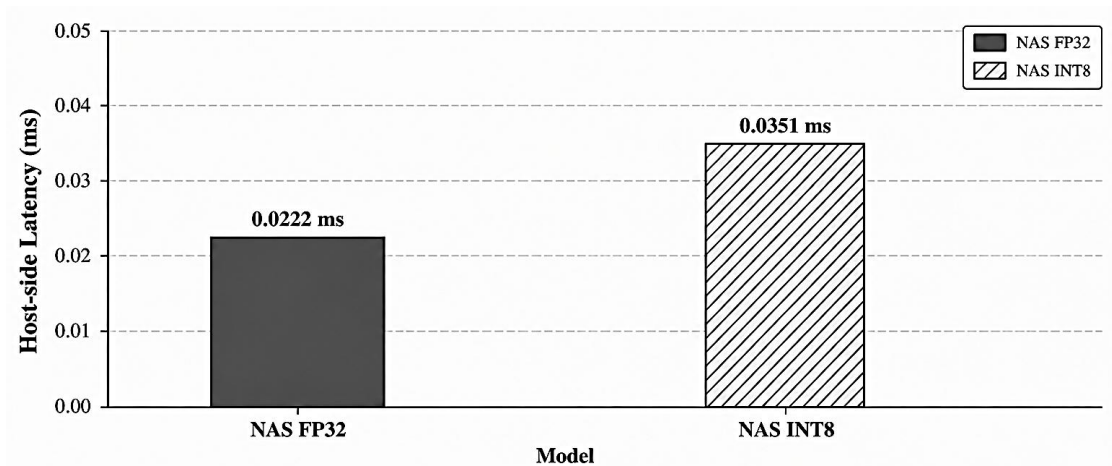


Figure. 8.3 Inference Latency Comparison

As Table 8.3 and Figure. 8.3 show, measured host-side latency comes to 0.0222 ms for the FP32 model and 0.0351 ms for the INT8 model - the INT8 version runs slightly slower in this development environment.

This result needs to be read with some care, since the latency measurements come from the host-side development environment rather than the target ESP32 itself, and floating-point hardware on the host

executes quite differently from a microcontroller. The current latency figures should therefore not be taken as a final measurement of ESP32 inference performance.

This underlines the importance of hardware-aware NAS: future evaluation should measure latency directly on the target platform and feed those measurements into the architecture-selection process.

8.7 EFFECT OF INT8 QUANTIZATION

The selected architecture was converted from FP32 to full-integer INT8 through post-training quantization, aiming to reduce the deployment footprint while keeping the learned architecture and classification capability intact.

The classification comparison shows quantization did not cost any reported overall accuracy - accuracy actually rose from 0.8838 (NAS-selected FP32) to 0.8899 (INT8), while macro-F1 moved from 0.6148 to 0.6188. These are empirical results from this particular evaluation and should not be read as a universal effect of INT8 quantization.

The clearest measurable benefit of quantization shows up in memory usage: as Table 8.2 indicates, estimated runtime memory drops to 10,869 bytes while the parameter count stays at 1,269. This makes quantization complementary to architecture search - NAS trims unnecessary architectural complexity, and quantization trims the representation cost of whatever network NAS selects.

This result demonstrates that architecture optimization and model quantization can be applied one after the other to improve deployment efficiency without touching the searched network topology.

8.8 ACCURACY, EFFICIENCY AND DEPLOYMENT TRADE-OFF

The overall results capture the central trade-off this project addresses: the manual baseline runs to 13,973 parameters, against just 1,269 for the NAS-selected network, while the NAS-selected FP32 and INT8 models still report higher accuracy than the manual baseline.

The selected model therefore offers a compact representation without giving up useful classification performance [26]. The Pareto-based search matters here because no architecture can be assumed optimal in macro-F1, model size and latency all at once - different candidates can sit in different positions across that performance-resource space.

Table 8.4 Overall NAS Performance and Deployment Characteristics

Characteristic	Reported Result
NAS Candidates Evaluated	50
Parameter Constraint	120,000
Selected Parameters	1,269
Baseline Parameters	13,973
Parameter Reduction	Approximately 11×
INT8 Serialized Size	8.23 kB
INT8 Estimated Runtime Memory	10,869 B
INT8 Accuracy	0.8899
INT8 Macro-F1	0.6188
FP32 Host-side Latency	0.0222 ms
INT8 Host-side Latency	0.0351 ms

As Table 8.4 summarizes, the main outcome is a much smaller architecture that still holds onto competitive classification performance - supporting the case for multi-objective architecture search in wearable ECG applications where memory and computation are tightly constrained.

Power needs to be read separately from these figures, though. A smaller network and integer computation could plausibly reduce

computational and memory-related energy consumption, but this experiment does not directly measure energy per inference, battery consumption or ESP32 power usage, so power efficiency cannot be claimed as an experimentally verified result of this phase.

8.9 DISCUSSION

These results show that the proposed NAS methodology can automatically find a highly compact ECG classifier from a constrained search space. The 1,269-parameter network is roughly eleven times smaller than the 13,973-parameter manually designed baseline, reaching 0.8838 accuracy in FP32 and 0.8899 after INT8 quantization.

The class-wise results also reveal a real limitation: strong performance on the Normal and Ventricular categories contrasts with the harder-to-classify minority Fusion and Supraventricular classes. This is exactly why macro-F1 remains an essential evaluation metric, and it points to more work still needed on minority-class recognition [2].

The memory results matter directly for the intended wearable deployment: INT8 quantization cuts estimated runtime memory from 43,476 bytes to 10,869 bytes while keeping the 1,269-parameter architecture, showing that quantization can meaningfully improve deployment efficiency once the architecture itself is optimized.

That said, this experimental scope carries two real limitations: latency is measured on the host development environment rather than directly on the ESP32, and classifier power consumption is neither measured nor optimized. This points to direct ESP32 deployment, target-device latency measurement and future power-aware NAS as the natural next stages [10].

Overall, the results support multi-objective NAS as a practical way to balance ECG classification performance against model compactness and inference efficiency. The next chapter pulls these findings together into the project conclusion and lays out the future scope - direct ESP32 deployment, hardware-aware latency evaluation, larger NAS search budgets and explicit power or energy optimization.

CHAPTER 9

CONCLUSION

This chapter pulls together the work carried out on multi-objective Neural Architecture Search for wearable ECG arrhythmia classification, summarizing the main outcomes of the proposed framework and identifying what still needs to happen to move from today's host-side evaluation to complete hardware-aware and power-aware edge deployment.

9.1 CONCLUSION

This project has developed a resource-aware neural architecture search framework for wearable ECG arrhythmia classification. Rather than relying entirely on manual neural-network design, the approach frames architecture selection as a multi-objective optimization problem that weighs classification performance and deployment-related resource requirements together. The ECG input is prepared as a 150×3 representation combining the normalized beat waveform with two causal RR-context features, and the architecture search runs within a predefined one-dimensional convolutional search space.

The NAS process uses evolutionary search - tournament selection, mutation and single-point crossover - with a 120,000-parameter constraint applied during architecture-space pre-pruning to keep oversized architectures out of expensive proxy training. Across the defined search configuration, 50 candidates were evaluated over four generations from a population of 10 architectures, with macro-F1 as the principal classification objective and parameter count and inference latency as resource-related objectives.

Pareto-based selection identified architectures representing different trade-offs between classification performance and resource efficiency. The final architecture - two compact convolutional stages, global average pooling, a 32-unit dense layer and a five-class softmax output - holds only 1,269 trainable parameters, against 13,973 in the manually designed baseline.

The experimental results show that this compact NAS architecture holds onto competitive classification performance. The manually designed FP32 baseline reached 0.8669 accuracy and 0.6203 macro-F1, the NAS-selected FP32 model reached 0.8838 accuracy and 0.6148 macro-F1, and after full-integer INT8 quantization the selected model reached 0.8899 accuracy and 0.6188 macro-F1. In short, the searched architecture cut parameter count by roughly eleven times while keeping balanced classification performance comparable.

Quantization further improved the deployment-oriented characteristics of the selected model: estimated runtime memory dropped from 43,476 bytes (FP32) to 10,869 bytes (INT8), roughly a fourfold reduction, and serialized model size fell from 9.5 kB to 8.23 kB. These results show architecture search and quantization playing complementary roles in cutting the resource requirements of a wearable ECG classifier.

The class-wise evaluation also highlighted how hard minority arrhythmia categories remain to recognize. Fusion recorded a particularly low F1-score, reinforcing why macro-F1 was preferred over raw accuracy during architecture selection - reducing model size and lifting overall accuracy do not by themselves guarantee uniform performance across every ECG class.

Overall, this project demonstrates that multi-objective NAS can identify compact neural architectures for wearable ECG classification. It

builds a foundation where predictive performance, model complexity and inference efficiency are weighed together, rather than treating classification accuracy as the only criterion. That said, the current implementation should be read within its experimental scope, since the reported latency is host-side and direct classifier power consumption has not been measured.

9.2 FUTURE SCOPE

The most immediate next step is deploying the selected INT8 model directly on the target ESP32. Right now, inference runs on the host-side or edge processing environment while the ESP32 handles ECG acquisition, packetisation, display and wireless communication; moving the quantised classifier onto the microcontroller through an embedded inference runtime such as TensorFlow Lite Micro would allow actual on-device memory and latency measurements to be taken [6].

The NAS framework can also be pushed toward genuine hardware-aware optimization [19]. Rather than relying on host-side latency or indirect architecture measures, measured ESP32 inference latency could be fed directly into the architecture-search process, aligning the selected architecture more closely with the actual deployment platform.

A further extension would explicitly fold power or energy consumption into the NAS objective [20]. This study treats power as an important deployment consideration without measuring energy-per-inference, battery life or ESP32 classifier power directly; future experiments could measure device-level power and optimize classification performance, latency and energy consumption together.

The search itself could also be expanded with a larger candidate budget, multiple random seeds and broader architecture variations. This

search evaluated 50 candidates from a single seed, so the selected network is best regarded as a strong configuration under that specific search budget, not a globally optimal architecture. A larger, repeated search could surface additional Pareto-efficient architectures with different accuracy, memory and latency characteristics.

Future work could also focus on improving minority-class performance, particularly for Fusion and Supraventricular categories. A larger, more representative dataset, better class-balancing strategies and broader temporal context could all help the classifier pick up patterns that are hard to spot from a single heartbeat. Extending the model to rhythm context across several consecutive beats is a further avenue worth exploring for arrhythmias whose diagnosis depends more on beat-to-beat timing.

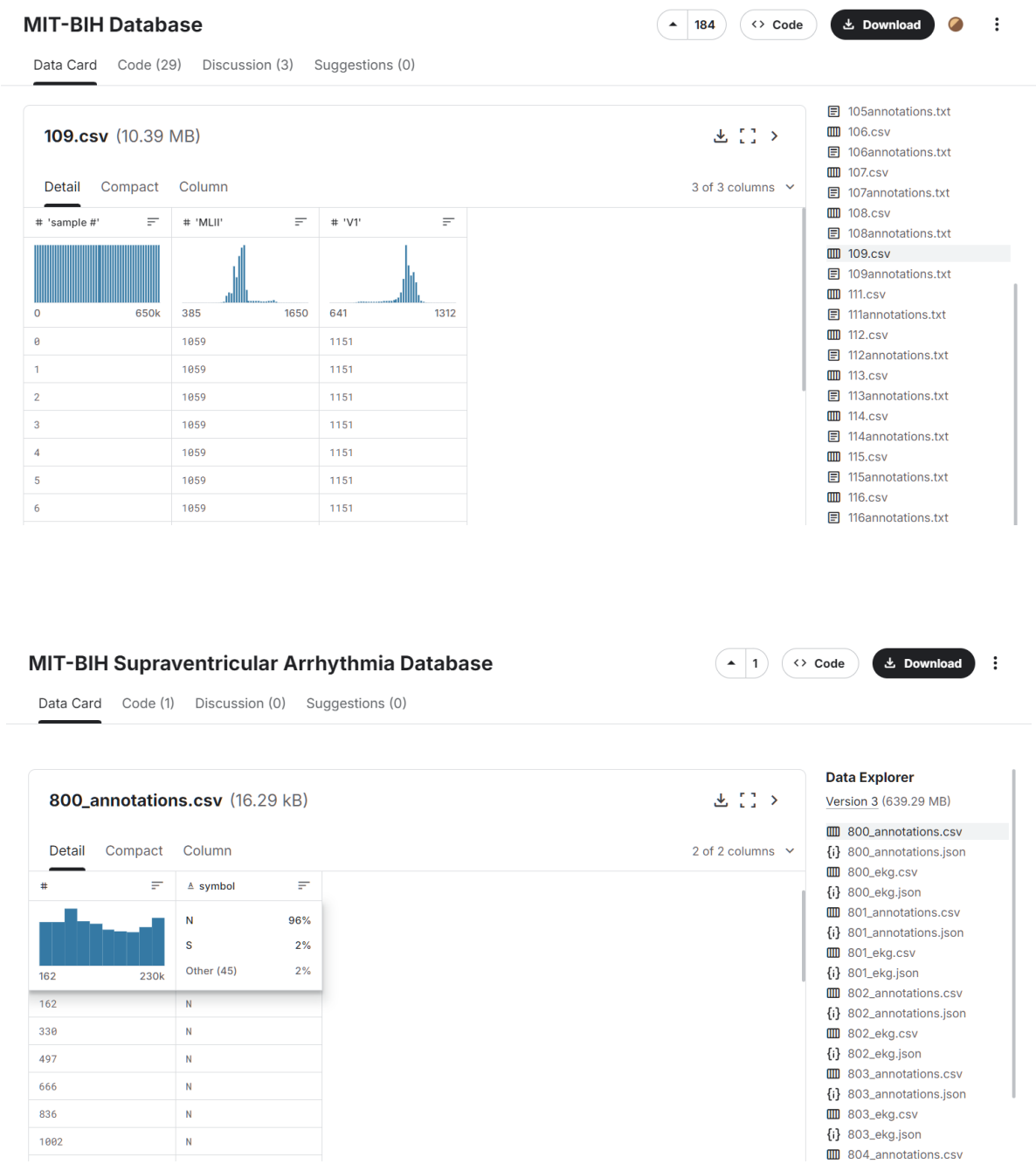
In the longer term, the proposed framework could grow from a single wearable node into a broader edge-based ECG monitoring system, with multiple devices performing local acquisition and inference while sharing selected performance information for centralized analysis - allowing architecture evaluation, deployment monitoring and model improvement to be studied across different operating conditions and hardware configurations.

Each of these extensions builds directly on the architecture established here. Direct ESP32 deployment, hardware-specific latency measurement, explicit power optimization, larger NAS searches and better minority-class recognition together represent a natural continuation of this methodology toward a more complete, hardware-aware, multi-objective ECG intelligence system.

APPENDICES

APPENDIX 1

ECG DATASET DETAILS USED IN THE PROJECT



Database Open Access

MIT-BIH Arrhythmia Database

George Moody, Roger Mark

Published: Feb. 24, 2005. Version: 1.0.0

MIT-BIH Arrhythmia Database expanded (Feb. 24, 2005, midnight)

The entire MIT-BIH Arrhythmia Database is now freely available on PhysioNet. Somewhat more than half of the database has been available here since PhysioNet's inception; the remainder has now been posted.

When using this resource, please cite the original publication:

Moody GB, Mark RG. The impact of the MIT-BIH Arrhythmia Database. IEEE Eng in Med and Biol 20(3):45-50 (May-June 2001). (PMID: 11446209)

Please include the standard citation for PhysioNet:

Cite Copy BibTeX

Pollard, T., Moody, B. E., Lehman, L., Gow, B., Fernandes, C., Xie, C., Johnson, A., Mark, R. G., & Heldt, T. (2026). PhysioNet as a global platform for biomedical research. Nature Health. <https://doi.org/10.1038/s44360-026-00096-z>. Available from: <https://rdcu.be/faatM>

Share



Access

Access Policy:

Anyone can access the files, as long as they conform to the terms of the specified license.

License (for files):

Open Data Commons Attribution License v1.0

Discovery

Background

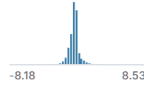
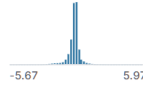
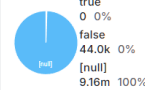
Since 1975, our laboratories at Boston's Beth Israel Hospital (now the Beth Israel Deaconess Medical Center) and at MIT have supported our own research into arrhythmia analysis and related subjects. One of the first major products of that effort was the MIT-BIH Arrhythmia Database, which we completed and began distributing in 1980. The database was the first generally available set of standard test material for evaluation of arrhythmia detectors, and has been used for that purpose as well as for basic research into cardiac dynamics at more than 500 sites worldwide. Originally, we distributed the database on 9-track half-inch digital tape at 800 and 1600 bpi, and on quarter-inch IRIG-format FM analog tape. In August, 1989, we produced a CD-ROM version of the database.

MIT-BIH Atrial Fibrillation Database

Data Card Code (2) Discussion (0) Suggestions (0)

04015_ekg.csv (204.06 MB)

Detail Compact Column

#	# ECG1	# ECG2	✓ qrs_annotation	Δ generic_annotation
index of the voltage at 250 samples per second	one channel from the EKG in millivolts	one channel from the EKG in millivolts	This will have a symbol if a doctor or tech saw a heartbeat peak here	Any other generic annotations made by doctors/techs
				
0	-0.275	-0.21		
1	-0.295	-0.2		
2	-0.31	-0.22		
3	-0.315	-0.225		
4	-0.3	-0.215		
5	-0.32	-0.22		

	A	B	C	D	E	F
1	ECG1	ECG2	qrs_annot	generic_annotation		
2	0	-0.275	-0.21			
3	1	-0.295	-0.2			
4	2	-0.31	-0.22			
5	3	-0.315	-0.225			
6	4	-0.3	-0.215		+	
7	5	-0.32	-0.22			
8	6	-0.3	-0.21			
9	7	-0.3	-0.195			
10	8	-0.29	-0.19			
11	9	-0.28	-0.205			
12	10	-0.295	-0.19			
13	11	-0.29	-0.21			
14	12	-0.305	-0.2 N			
15	13	-0.315	-0.2			
16	14	-0.32	-0.215			
17	15	-0.295	-0.215			
18	16	-0.295	-0.215			
19	17	-0.305	-0.195			
20	18	-0.275	-0.205			
21	19	-0.25	-0.2			
22	20	-0.23	-0.195			
23	21	-0.19	-0.2			
24	22	-0.175	-0.185			
25	23	-0.165	-0.16			
26	24	-0.17	-0.12			
27	25	-0.14	-0.13			
28	26	-0.145	-0.13			
29	27	-0.12	-0.125			
30	28	-0.12	-0.145			

This project primarily uses the MIT-BIH Arrhythmia Database, for beat-level ECG classification, with the MIT-BIH Supraventricular Arrhythmia Database and MIT-BIH Atrial Fibrillation Database providing additional ECG resources for extended analysis.

The MIT-BIH Arrhythmia Database contains 48 half-hour two-channel ECG recordings from 47 subjects, sampled at 360 samples per second, with cardiologist-annotated beats; it is the main dataset used for the classification experiment.

The MIT-BIH Supraventricular Arrhythmia Database, contains 78 half-hour recordings and supplies additional examples of supraventricular arrhythmias.

The MIT-BIH Atrial Fibrillation Database, contains 25 long-term recordings and is useful for future rhythm-level analysis involving longer temporal context.

These ECG recordings are converted into beat-level samples and prepared as a 150×3 input containing the normalized ECG waveform and two causal RR-context features. Since the class distribution is highly imbalanced, macro-F1 is used during NAS evaluation.

APPENDIX 2

EVOLUTIONARY NAS ALGORITHM USED IN THE PROJECT

The proposed architecture search uses an evolutionary NAS approach to automatically find a compact 1-D CNN for ECG classification, combining parameter pre-pruning, proxy training, multi-objective evaluation, Pareto selection, tournament selection, crossover and mutation.

The NAS runs a population of 10 architectures over 4 generations, evaluating 50 candidates in total. Each candidate goes through 4 epochs of proxy training, and architectures above 120,000 parameters are removed. Surviving architectures are evaluated on macro-F1, parameter count and inference latency, with Pareto ranking and weighted fitness guiding tournament selection and evolutionary reproduction.

New architectures are generated and evaluated across successive generations through crossover and mutation. The final selected architecture holds just 1,269 trainable parameters, made up of two convolutional stages, global average pooling, a 32-unit dense layer and a five-class softmax output.

This selected model is then converted to INT8 for deployment, reducing estimated runtime memory from 43,476 bytes to 10,869 bytes while reaching 0.8899 accuracy and a 0.6188 macro-F1 score.



SUSTAINABLE DEVELOPMENT GOALS MAPPING

The Sustainable Development Goals (SDGs) are a set of 17 global goals meant to serve as a blueprint for a better, more sustainable future. The United Nations General Assembly set these goals in 2015, with 2030 as the target year, after 195 nations agreed on this shared blueprint for change. The present project maps onto the goals listed below.

Questions	Answers
Which SDGs does the project directly address?	SDG 3 – Good Health and Well-being (primary); SDG 9 – Industry, Innovation and Infrastructure; SDG 12 – Responsible Consumption and Production.
What strategies or actions are being implemented to achieve these goals?	For SDG 3, the project develops a wearable ECG arrhythmia classification system using a compact neural network to support continuous ECG analysis. For SDG 9, multi-objective NAS, evolutionary optimization, Pareto selection and edge-oriented model design are used to develop an innovative resource-aware AI system. For SDG 12, model parameters, memory requirements and inference latency are reduced through architecture optimization and INT8 Quantization, supporting more efficient computational resource usage.
How is progress measured and reported in relation to the SDGs?	Progress is measured using ECG classification accuracy and macro-F1 for health-related performance, together with parameter count, serialized model size, estimated runtime memory and inference latency for technological

	efficiency. The reported INT8 model contains 1,269 parameters, occupies 8.23 kB in serialized form and requires an estimated 10,869 bytes of runtime memory.
How were these goals identified as relevant to the project's objectives?	SDG 3 is directly related because the project addresses automated ECG arrhythmia classification and wearable health monitoring, aligning with the UN focus on prevention and treatment of non-communicable diseases. SDG 9 is relevant because the project applies AI, neural architecture search and embedded/edge technologies to develop an innovative and resource-aware monitoring system; SDG 9 specifically includes sustainable infrastructure, technological upgrading and innovation. SDG 12 is relevant because the project emphasizes efficient use of computational resources through compact architecture design and Quantization, consistent with the goal of improving resource efficiency.
Are there any partnerships or collaborations in place to enhance this impact?	The present work is developed as an academic engineering project using publicly available ECG resources and computational tools. The framework can be extended through future collaboration with healthcare institutions, biomedical researchers, embedded-system developers and technology organizations for larger-scale clinical validation and on-device deployment.

Through these mappings, the project shows how AI-based wearable ECG analysis can support sustainable development by combining health-oriented monitoring with efficient computational design. Multi-objective NAS lets classification performance be weighed alongside model size and inference requirements, while INT8 quantization further trims the deployment footprint.

REFERENCES

1. Dubatovka, A. and Buhmann, J.M. (2022) ‘Automatic Detection of Atrial Fibrillation from Single-Lead ECG Using Deep Learning of the Cardiac Cycle’, BME Frontiers, Vol.2022, Article 9813062.
2. Ebrahimi, Z., Loni, M., Daneshtalab, M. and Gharehbaghi, A. (2020) ‘A Review on Deep Learning Methods for ECG Arrhythmia Classification’, Expert Systems with Applications: X, Vol.7, Article 100033.
3. Hannun, A.Y., Rajpurkar, P., Haghpanahi, M., Tison, G.H., Bourn, C., Turakhia, M.P. and Ng, A.Y. (2019) ‘Cardiologist-level Arrhythmia Detection and Classification in Ambulatory Electrocardiograms Using a Deep Neural Network’, Nature Medicine, Vol.25, pp.65-69.
4. Acharya, U.R., Oh, S.L., Hagiwara, Y., Tan, J.H., Adam, M., Gertych, A. and Tan, R.S. (2017) ‘A Deep Convolutional Neural Network Model to Classify Heartbeats’, Computers in Biology and Medicine, Vol.89, pp.389-396.
5. Kiranyaz, S., Ince, T. and Gabbouj, M. (2016) ‘Real-Time Patient-Specific ECG Classification by 1-D Convolutional Neural Networks’, IEEE Transactions on Biomedical Engineering, Vol.63, No.3, pp.664-675.
6. Lin, J., Chen, W.M., Lin, Y., Cohn, J., Gan, C. and Han, S. (2020) ‘MCUNet: Tiny Deep Learning on IoT Devices’, Advances in Neural Information Processing Systems, Vol.33.
7. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H. and Kalenichenko, D. (2018) ‘Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference’, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp.2704-2713.
8. Liu, H., Simonyan, K. and Yang, Y. (2019) ‘DARTS: Differentiable Architecture Search’, Proc. International Conference on Learning Representations (ICLR).
9. Pham, H., Guan, M.Y., Zoph, B., Le, Q.V. and Dean, J. (2018) ‘Efficient Neural Architecture Search via Parameter Sharing’, Proc. International Conference on Machine Learning (ICML), Vol.80, pp.4092-4101

10. Cai, H., Zhu, L. and Han, S. (2019) ‘ProxylessNAS: Direct Neural Architecture Search on Target Task and Hardware’, Proc. International Conference on Learning Representations (ICLR).
11. Han, S., Mao, H. and Dally, W.J. (2016) ‘Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding’, Proc. International Conference on Learning Representations (ICLR).
12. Cai, H., Gan, C., Wang, T., Zhang, Z. and Han, S. (2020) ‘Once for All: Train One Network and Specialize It for Efficient Deployment’, Proc. International Conference on Learning Representations (ICLR).
13. Kachuee, M., Fazeli, S. and Sarrafzadeh, M. (2018) ‘ECG Heartbeat Classification: A Deep Transferable Representation’, Proc. IEEE International Conference on Healthcare Informatics (ICHI), pp.443-444.
14. Kim, S., Chon, S., Kim, J.K., Kim, J., Gil, Y. and Jung, S. (2022) ‘Lightweight Convolutional Neural Network for Real-Time Arrhythmia Classification on Low-Power Wearable Electrocardiograph’, Proc. IEEE Engineering in Medicine and Biology Conference (EMBC), pp.1915-1918.
15. Sabor, N., Gendy, G., Mohammed, H., Wang, G. and Lian, Y. (2022) ‘Robust Arrhythmia Classification Based on QRS Detection and a Compact 1D-CNN for Wearable ECG Devices’, IEEE Journal of Biomedical and Health Informatics, Vol.26, No.12, pp.5918-5929.
16. Dong, J.D., Cheng, A.C., Juan, D.C., Wei, W. and Sun, M. (2018) ‘DPP-Net: Device-aware Progressive Search for Pareto-optimal Neural Architectures’, Proc. European Conference on Computer Vision (ECCV), pp.517-531.
17. Lu, B., Yang, J., Jiang, W., Shi, Y. and Ren, S. (2021) ‘One Proxy Device Is Enough for Hardware-Aware Neural Architecture Search’, Proceedings of the ACM on Measurement and Analysis of Computing Systems, Vol.5, No.3, Article 34, pp.1-34.
18. He, Y., Lin, J., Liu, Z., Wang, H., Li, L.J. and Han, S. (2018) ‘AMC: AutoML for Model Compression and Acceleration on Mobile Devices’, Proc. European Conference on Computer Vision (ECCV), pp.784-800.
19. Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A. and Le, Q.V. (2019) ‘MnasNet: Platform-Aware Neural Architecture Search for Mobile’, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp.2820-2828.
20. Wang, K., Liu, Z., Lin, Y., Lin, J. and Han, S. (2019) ‘HAQ: Hardware-Aware Automated Quantization with Mixed Precision’, Proc. IEEE/CVF

- Conference on Computer Vision and Pattern Recognition (CVPR), pp.8612-8620.
21. Rajpurkar, P., Hannun, A.Y., Haghpanahi, M., Bourn, C. and Ng, A.Y. (2017) ‘Cardiologist-Level Arrhythmia Detection with Convolutional Neural Networks’, arXiv:1707.01836.
 22. Ribeiro, A.H., Ribeiro, M.H., Paixão, G.M.M., Oliveira, D.M., Gomes, P.R., Canazart, J.A., Ferreira, M.P.S., Andersson, C.R., Macfarlane, P.W., Meira Jr., W., Schön, T.B. and Ribeiro, A.L.P. (2020) ‘Automatic Diagnosis of the 12-Lead ECG Using a Deep Neural Network’, Nature Communications, Vol.11, Article 1760.
 23. Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H. (2017) ‘MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications’, arXiv:1704.04861.
 24. Howard, A., Sandler, M., Chu, G., Chen, L.C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q.V. and Adam, H. (2019) ‘Searching for MobileNetV3’, Proc. IEEE/CVF International Conference on Computer Vision (ICCV), pp.1314-1324.
 25. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.C. (2018) ‘MobileNetV2: Inverted Residuals and Linear Bottlenecks’, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp.4510-4520.
 26. Tan, M. and Le, Q.V. (2019) ‘EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks’, Proc. International Conference on Machine Learning (ICML), Vol.97, pp.6105-6114.